

# Master Go, Vue, and Git

By Rebuilding the PEACE SME Grant Portal

A practical, chapter-by-chapter workbook generated from the application specification.



# Master Go, Vue, and Git by Rebuilding the PEACE SME Grant Portal

This guide is a chapter-by-chapter book for converting the existing Flask-based PEACE SME Grant Portal into a Go backend and a Vue 3 frontend while learning Git through disciplined, real project work. It is written for a coder who is new to Go: every backend concept is introduced theoretically, then immediately attached to a feature from the portal.

The source of truth for the application behavior is `Claude.md`. Every chapter ties a technical concept to one part of the portal: authentication, applicant profiles, grants, admin reports, HFC fraud scoring, uploads, queues, caching, bilingual UI, and deployment.

## Learning Goals

By the end of the book, you should be able to:

- Design and implement production Go HTTP services.
- Model PostgreSQL schemas and JSONB data in Go.
- Build secure authentication with JWT and bcrypt.
- Use middleware for auth, geo-blocking, access control, logging, and recovery.
- Implement Redis caching, concurrency controls, and background jobs.
- Integrate S3-compatible object storage and Brevo email delivery.
- Build a Vue 3 SPA using Composition API, Vue Router, Axios, Tailwind CSS, and Chart.js.
- Preserve an existing API contract while replacing the backend implementation.
- Use Git branches, commits, diffs, merges, tags, rebases, and pull requests as part of daily engineering work.

## Recommended Repository Layout

The Go backend can evolve from the current minimal module into this structure:

```
cmd/server/  
  main.go  
internal/  
  app/  
  auth/  
  cache/  
  config/  
  db/  
  httpx/  
  security/  
  user/  
  business/  
  grant/  
  admin/  
  report/  
  hfc/  
  status/  
  content/  
  storage/  
  mail/  
migrations/  
frontend/  
  src/
```

## Chapters

Use the Concept Index when you want to study by topic instead of chapter order.

- 1 Read the Existing System Like an Engineer
  - 2 Git Foundations for a Rewrite
  - 3 Go Project Structure and HTTP Server
  - 4 Configuration, Environment, and Application Toggles
  - 5 PostgreSQL, Migrations, and Data Modeling
  - 6 Authentication, JWT, bcrypt, and Middleware
  - 7 Applicant Registration, Login, and Business Profiles
  - 8 Grant Applications and Workflow Rules
  - 9 Uploads, S3-Compatible Storage, and Media References
  - 10 Redis, Caching, Access Slots, and Background Jobs
  - 11 Admin, Reports, CSV, and Database Browser
  - 12 HFC Fraud Detection and Approval Authority
  - 13 Vue 3 Frontend Foundations
  - 14 Vue Applicant and Admin Interfaces
  - 15 Testing, Debugging, and API Compatibility
  - 16 Deployment, Release Git, and Mastery Roadmap
- 
- 1 Go Beginner Primer Through Portal Features
  - 2 Project Parallel Workbook
  - 3 Vue and Git Practice Lab

## How to Work Through the Book

Use one branch per chapter:

```
git checkout -b chapter-03-go-server
```

Make small commits:

```
git add .
git commit -m "Add Go server bootstrap"
```

At the end of each chapter, compare your work against the contract in `Claude.md`. The goal is not merely to make something work. The goal is to reproduce the same API paths, request shapes, response shapes, business rules, and frontend behavior with a Go + Vue implementation you understand deeply.

## The Learning Pattern Used in Every Chapter

Each chapter should be read in this order:

- 1 Theory: learn the language or engineering concept.
- 2 Application parallel: identify where the PEACE SME portal needs that concept.
- 3 Backend implementation: build the Go package, handler, service, repository, or worker piece.
- 4 Frontend implementation: connect the Vue screen or component to the matching API behavior.
- 5 Git practice: save the work as a small, reviewable commit.
- 6 Mastery check: prove you can explain and modify the concept without copying code.

For example, you do not learn Go structs in isolation. You learn structs by modeling users, businesses, and grants. You do not learn Vue reactivity in isolation. You learn reactivity by building the grant form where changing

contribution type reveals cash or in-kind fields. You do not learn Git branches abstractly. You learn branches by creating `feature/grant-access-whitelist`, reviewing its diff, and merging it after tests pass.

# Concept Index

This index maps the major skills to the chapters where they are practiced.

## Go

| Concept                        | Chapters        |
|--------------------------------|-----------------|
| Project layout                 | 3               |
| cmd and internal packages      | 3               |
| HTTP handlers                  | 3, 7, 8, 11     |
| JSON request/response helpers  | 3               |
| Configuration loading          | 4               |
| Environment toggles            | 4, 7, 8, 10, 12 |
| Interfaces                     | 8, 10, 12       |
| Dependency injection           | 3, 7            |
| Context propagation            | 5, 6            |
| Error handling                 | 3, 6, 15        |
| Middleware                     | 6               |
| JWT                            | 6               |
| bcrypt                         | 6               |
| Repository pattern             | 5               |
| Transactions                   | 5, 12           |
| Background worker boundaries   | 10, 12          |
| Testing                        | 15              |
| Beginner Go concept map        | 17              |
| Feature-by-feature Go practice | 18              |

## PostgreSQL

| Concept                         | Chapters |
|---------------------------------|----------|
| Schema preservation             | 5        |
| Migrations                      | 5, 16    |
| Foreign keys and cascade delete | 5        |
| Unique constraints              | 7, 8     |
| JSONB                           | 5, 8     |
| Nullable columns                | 5        |
| Indexes                         | 5, 11    |
| Paginated queries               | 11       |
| Filtered reports                | 11       |
| Defensive dynamic SQL           | 11       |
| Audit tables                    | 12       |
| Feature-to-table mapping        | 18       |

## Vue

| Concept                        | Chapters |
|--------------------------------|----------|
| Vue 3 SPA structure            | 13       |
| Composition API                | 13, 14   |
| ref, reactive, computed, watch | 13, 14   |
| Vue Router                     | 13       |
| Route guards                   | 13       |
| Axios clients                  | 13       |
| localStorage token handling    | 13       |
| Bilingual English/Urdu UI      | 13, 14   |
| RTL layout                     | 13, 14   |
| Large forms                    | 14       |
| Repeating form rows            | 14       |
| Admin tables                   | 14       |
| Charts                         | 14       |
| Loading/error/empty states     | 14       |
| Vue practice labs              | 19       |
| Concept-to-screen mapping      | 19       |

## Git

| Concept                | Chapters |
|------------------------|----------|
| Status and diffs       | 1, 2     |
| Branches               | 2        |
| Small commits          | 2        |
| Staging area           | 2        |
| Commit messages        | 2        |
| Reading history        | 2        |
| Merge                  | 2        |
| Rebase                 | 2        |
| Conflict resolution    | 2        |
| Tags                   | 16       |
| Release branches       | 16       |
| Hotfix branches        | 16       |
| Bisect                 | 15       |
| Feature branch labs    | 19       |
| Commit review workflow | 19       |

## Security

| Concept               | Chapters |
|-----------------------|----------|
| Password verification | 6        |

| Concept                    | Chapters |
|----------------------------|----------|
| Bearer auth                | 6, 13    |
| Admin roles                | 6, 12    |
| Approver-only endpoints    | 6, 12    |
| Geo-blocking               | 6, 10    |
| Access slots               | 10       |
| Upload ownership checks    | 9        |
| Query token exports        | 11       |
| SQL injection prevention   | 11       |
| End-to-end auth boundaries | 18, 19   |

## Production Engineering

| Concept                       | Chapters |
|-------------------------------|----------|
| API compatibility             | 1, 15    |
| Caching                       | 10, 11   |
| Redis TTLs                    | 10       |
| S3-compatible uploads         | 9        |
| Email jobs                    | 10, 12   |
| HFC scoring                   | 12       |
| Auditability                  | 12       |
| Docker builds                 | 16       |
| Migrations at deploy time     | 16       |
| Rollback planning             | 16       |
| Release tagging               | 16       |
| End-to-end project milestones | 18       |
| Full capstone workflow        | 19       |



# Chapter 1: Read the Existing System Like an Engineer

## Purpose

Before writing Go or Vue code, learn to extract behavior from an existing application. The PEACE SME Grant Portal is not just a set of pages. It is a workflow:

- 1 Applicant registration and login.
- 2 Business profile entry.
- 3 Document and media uploads.
- 4 Admin eligibility decisions.
- 5 Whitelisted grant application submission.
- 6 HFC fraud scoring.
- 7 Approval by an approving authority.
- 8 Notifications, reports, exports, FAQs, updates, and operational controls.

The rewrite succeeds only if these behaviors remain compatible.

For a beginner, this chapter teaches an important professional habit: do not start a rewrite by opening an editor and inventing code. Start by reading the system until you can describe what must remain true after the rewrite.

## Theory: What Is a Rewrite?

A rewrite is not "build a new app that looks similar." A rewrite is a controlled replacement of implementation while preserving required behavior.

In this project:

- Old implementation: Flask backend plus Vue frontend.
- New implementation: Go backend plus Vue 3 frontend.
- Stable contract: PostgreSQL schema, API paths, JSON shapes, auth behavior, business rules, and user workflows.

Think of the rewrite as changing the engine of a running system. The outside controls should still work. The frontend expects the same endpoints. The database stores the same business facts. Admins expect the same reports. Applicants expect the same workflow.

## Application Parallel: The First Task You Are Really Building

Your first project task is not a route or a page. It is a map.

Build a map that answers:

- Which frontend view calls which endpoint?
- Which endpoint uses which service?
- Which service reads or writes which table?
- Which business rule protects that operation?
- Which Git branch will contain that work?

Example:

| User action              | Vue view               | API endpoint                             | Go package                     | Tables                      | Rule                       |
|--------------------------|------------------------|------------------------------------------|--------------------------------|-----------------------------|----------------------------|
| Applicant logs in        | UserLogin.vue          | POST /api/login                          | internal/user, internal/auth   | users                       | blocked users cannot login |
| Applicant saves business | SmeBusinessProfile.vue | POST /api/business                       | internal/business              | businesses                  | one business per user      |
| Admin approves grant     | AdminGrantDetail.vue   | POST /api/admin/grants/<user_id>/approve | internal/grant, internal/admin | grants, grant_approval_logs | approver role required     |

This map becomes your implementation checklist.

## What to Read

Start with:

- Claude.md
- README.md
- go.mod
- cmd/server/main.go

The current Go service is only a health-check stub. That is useful: the project is clean enough to grow in deliberate layers.

## System Map

The original application has these major boundaries:

| Boundary         | Flask module                         | Go package to create             | Vue area                       |
|------------------|--------------------------------------|----------------------------------|--------------------------------|
| Auth             | auth_service.py, security.py         | internal/auth, internal/security | Login views, route guards      |
| Users            | user_service.py                      | internal/user                    | Registration, login, dashboard |
| Business profile | user_service.py                      | internal/business                | SmeBusinessProfile.vue         |
| Grants           | grant_service.py                     | internal/grant                   | SmeGrantApplication.vue        |
| Reports          | report_service.py                    | internal/report                  | Admin report views             |
| Admin            | admin_service.py                     | internal/admin                   | Admin dashboard and tools      |
| HFC              | hfc_service.py, hfc_admin_service.py | internal/hfc                     | HFC admin views                |
| Applicant status | status_service.py                    | internal/status                  | Applicant status admin view    |
| Content          | updates, FAQs                        | internal/content                 | Landing page, FAQ bot          |
| Storage          | s3_service.py                        | internal/storage                 | Upload components              |
| Mail             | mail_service.py                      | internal/mail                    | Approval side effects          |

## API Contract Thinking

The Vue frontend depends on:

- Exact endpoint paths under /api.
- Exact localStorage keys: userToken, adminToken, language.

- Exact pagination shapes such as {data, total, page, per\_page}.
- JWTs sent as Authorization: Bearer <token>.
- Admin JWT fields such as is\_admin and is\_approver.
- English stored values even when the UI is Urdu.

When converting Flask to Go, you are not free to invent a cleaner public contract until you also migrate the frontend.

## Contract vs Implementation

The contract is what other parts of the system depend on. The implementation is how you satisfy it.

| Contract                                           | Implementation detail                           |
|----------------------------------------------------|-------------------------------------------------|
| POST /api/login returns {token, user_id, language} | Go may use net/http, chi, or another router     |
| Passwords use bcrypt                               | Repository structure is up to you               |
| JWT uses HS256 and JWT_SECRET_KEY                  | Token service can be written as a small package |
| Reports return {data, total, page, per_page}       | SQL can be optimized later                      |
| GRANT_REQUIRE_SELECTION=1 gates grant submission   | The check can live in GrantService              |

As a learner, this distinction prevents a common mistake: changing the public behavior while trying to improve the internal design.

## Beginner Go Lens

This system will teach Go in layers:

- 1 Structs model data such as users and grants.
- 2 Functions and methods implement operations such as login and approval.
- 3 Interfaces separate side effects such as S3 and email from business logic.
- 4 Errors make invalid states explicit.
- 5 Context carries request cancellation and identity.
- 6 Goroutines and workers handle slow work such as email and HFC scoring.

Do not try to master all Go features before building. Learn each feature when the application gives you a reason to use it.

## Exercise

Create a file named guide/app-contract-notes.md while studying. Capture:

- Every endpoint you plan to implement.
- Every table you plan to migrate.
- Every feature toggle.
- Every auth rule.
- Every response shape that the frontend depends on.

Add one more section called "parallel learning notes." For every feature, write the Go concept it teaches:

| Feature          | Go concept                     | Vue concept              | Git concept                         |
|------------------|--------------------------------|--------------------------|-------------------------------------|
| Login            | structs, bcrypt, JWT, errors   | form state, localStorage | small auth branch                   |
| Business profile | validation, repository methods | reactive form            | commit create and update separately |

| Feature | Go concept                    | Vue concept               | Git concept                  |
|---------|-------------------------------|---------------------------|------------------------------|
| Reports | SQL filters, maps, pagination | table state, query params | review staged diff carefully |

## Git Practice

Initialize your learning branch:

```
git status
git checkout -b chapter-01-system-reading
git add guide
git commit -m "Add system reading notes"
```

Use `git diff --staged` before committing. This teaches you to review your own work before Git records it.

## Mastery Check

You understand this chapter when you can explain:

- Why the Go rewrite must preserve the Flask API contract.
- Which modules belong in backend packages.
- Which parts of the frontend are public, applicant-only, and admin-only.
- Why `Claude.md` is a behavioral specification, not just documentation.

# Chapter 2: Git Foundations for a Rewrite

## Purpose

Git is not separate from the rewrite. It is the safety system that lets you change a large application without losing track of intent.

## Theoretical Background

### Git as a Directed Acyclic Graph (DAG)

Git stores history not as a list of changes, but as a Directed Acyclic Graph (DAG) of commit objects. Each commit contains:

- A cryptographic SHA-1 hash (identifying the commit uniquely based on file contents, metadata, parent hashes, author, and timestamp).
- A pointer to zero or more parent commits.
- A snapshot of the entire directory tree (not delta changes).

Understanding this graph structure helps you visualize operations:

- A **Branch** is simply a mutable pointer to a specific commit.
- **HEAD** is a pointer to the currently checked-out commit or branch.

### The Three-State Architecture

Git projects consist of three primary zones:

- 1 **Working Directory (Workspace):** The local sandbox where you edit files on disk.
- 2 **Staging Area (Index):** A binary file tracking the changes prepared for the next commit. This acts as a preparation area.
- 3 **Commit History (Repository/Git Directory):** The permanent metadata database storing the commit graph (`.git/objects/`).

```
[ Working Directory ] --- git add ---> [ Staging Area (Index) ] --- git commit ---> [ Commit History (.git) ]  
[                   ] <--- checkout --- [                   ] <--- commit ----- [                   ]
```

### Merging vs. Rebasing

- **Merge** (`git merge`): Integrates two branches by creating a special "merge commit" that has two parent commits. This preserves the exact historical timeline and order of changes, but can make the history cluttered with merge commits.
- **Rebase** (`git rebase`): Rewrites commit history by moving the base of your branch to a new starting point. It applies your commits sequentially as new commits on top of the target branch. This creates a clean, linear history but alters the SHA-1 hashes of the original commits.

[!WARNING] Never rebase commits that have been pushed to a shared public repository. Doing so rewrites history that others depend on.

### External Resources

- Git Book - Git Internals
- Git Book - Git Branching and Rebasing

## Core Workflow

Use this loop for every chapter:

```
git status
git checkout -b chapter-XX-short-name
git diff
git add <files>
git diff --staged
git commit -m "Describe the completed behavior"
```

Good commits are small, reviewable, and named after behavior:

- Add Go configuration loader
- Implement user login endpoint
- Add Vue admin route guards
- Add HFC scoring tests

Avoid vague messages:

- updates
- fix
- changes

## Practical Examples

### Example 1: Creating and Merging a Feature Branch

To start a new feature and safely integrate it back into the main branch:

```
# 1. Ensure you are on main and up to date
git checkout main
git pull origin main

# 2. Create and switch to the new feature branch
git checkout -b feature/auth-service

# 3. (After making changes to internal/auth/auth.go)
# Check what has changed in the working directory
git status

# 4. Stage your changes
git add internal/auth/auth.go

# 5. Review the staged diff before committing
git diff --staged

# 6. Commit the staged changes
git commit -m "Implement authentication service skeleton"

# 7. Merge the changes back into main
git checkout main
git merge feature/auth-service
```

### Example 2: Interactive Rebase to Clean Up Commits

Before submitting a pull request, you may want to squash minor "fixup" commits into a single logical commit:

```
# Start an interactive rebase for the last 3 commits
git rebase -i HEAD~3
```

This will open an editor showing:

```
pick a1b2c3d Add user registration validator
pick d4e5f6g Fix typo in registration validator
pick h7i8j9k Add tests for registration validation
```

To squash the typo fix into the main commit, change the file to:

```
pick a1b2c3d Add user registration validator
squash d4e5f6g Fix typo in registration validator
pick h7i8j9k Add tests for registration validation
```

Save and close the editor. Git will combine the first two commits and prompt you for a new commit message.

### Example 3: Resolving a Merge Conflict

If you and another developer edit the same line of a file, Git will report a conflict:

```
Auto-merging cmd/server/main.go
CONFLICT (content): Merge conflict in cmd/server/main.go
Automatic merge failed; fix conflicts and then commit the result.
```

Opening `cmd/server/main.go` will show the conflict markers:

```
<<<<<< HEAD
    addr := ":8080"
=====
    addr := ":9000"
>>>>>> feature/custom-port
```

To resolve this conflict:

- 1 Discuss or decide which value is correct (or combine them).
- 2 Delete the conflict markers (`<<<<<<`, `=====`, `>>>>>>`) and edit the code to the desired state:

```
    addr := ":9000"
```

- 1 Stage the resolved file:

```
git add cmd/server/main.go
```

- 1 Complete the merge commit:

```
git commit -m "Resolve merge conflict, standardizing on port 9000"
```

## Reading History

Use:

```
git log --oneline --decorate --graph --all
git show <commit>
git diff main...HEAD
```

`git show` answers "what changed in this commit?" `git diff main...HEAD` answers "what does this branch change compared with main?"

## Handling Mistakes

Unstage a file:

```
git restore --staged path/to/file
```

Discard changes in one file only when you are sure:

```
git restore path/to/file
```

Inspect before undoing:

```
git diff path/to/file
```

## Mastery Check

You understand this chapter when you can:

- Create a branch per chapter.
- Review staged changes before committing.
- Explain the difference between working tree, staging area, and commit history.
- Recover from accidental staging.
- Resolve a simple conflict without panic.



# Chapter 3: Go Project Structure and HTTP Server

## Purpose

Replace the stub server with a production-shaped Go application. In this chapter, we will learn Go's dependency management, package visibility, custom data types, and pointers. We will apply these concepts by defining **every single core struct** used throughout the database, configuration, and API layers of the PEACE SME Grant Portal.

## Foundational Concepts Explained Simply

### 1. Go Modules (go mod)

In Go, a **Module** is a collection of Go packages versioned together. The `go.mod` file at the root of your project defines the module name and tracks dependencies:

- `go mod init <name>` initializes a module.
- `go get <package-path>` downloads a third-party dependency.
- `go.sum` stores cryptographic hashes of dependencies to ensure builds are reproducible and secure.

## Deep Dive: Go Modules & Dependency Tracking

### In-Depth Explanation

Before the introduction of Modules, Go developers were forced to place all source code inside a single global workspace directory named `$GOPATH`. This made project isolation and version pinning extremely difficult. Go Modules solved this by introducing local dependency manifests:

- `go.mod`: Declares the module's path identity (e.g. `module peace-sme-go`), the required Go language specification version, and a list of dependent packages with pinned semantic versions.
- `go.sum`: Records the checksums of all direct and indirect dependencies. If a hacker alters the dependency package on a mirror server, Go's checksum validation will fail-fast, preventing code injection.

## Sandbox Program: Module Manifest Simulation

Here is a simulation demonstrating how Go models depend on modules:

```
package main

import (
    ■ "fmt"
    ■ "strings"
)

// Dependency models a required library recorded in go.mod
type Dependency struct {
    ■ Path    string
    ■ Version string
}

func main() {
    ■ // Simulate the content of a go.mod file
    ■ moduleName := "peace-sme-go"
    ■ deps := []Dependency{
    ■ ■ {Path: "github.com/jackc/pgx/v5", Version: "v5.5.0"},
    ■ ■ {Path: "github.com/redis/go-redis/v9", Version: "v9.3.0"},
    ■ }

    ■ fmt.Printf("Module Name: %s\n", moduleName)
    ■ fmt.Println("--- Direct Dependencies ---")
    ■ for _, dep := range deps {
    ■ ■ fmt.Printf("Import: %-30s | Version: %s\n", dep.Path, dep.Version)
    ■ }
}
```

```
}
```

## 2. Packages and Visibility

Go groups files in the same directory into a **Package**. Go uses a simple rule for code visibility:

- **Exported (Public):** If a variable, function, struct, or field name begins with a **Capital Letter** (e.g., UserID, ApproveGrant), it is public and can be accessed by other packages importing it.
- **Unexported (Private):** If it begins with a **lowercase letter** (e.g., dbStatus, findUser), it is private and accessible only within its own package.

## Deep Dive: Package Layout, Imports & Visibility

### In-Depth Explanation

Go enforces a clean directory-to-package mapping:

- 1 Every file in a directory must belong to the **same package** (with testing files as a slight exception).
- 2 Package names should be lowercase, singular nouns (e.g. auth, not authenticator or AuthServices).
- 3 To consume functions from another package, you use the `import` statement followed by the module path and package name:

```
import "peace-sme-go/internal/config"
```

## Sandbox Program: Visibility and Namespace Rules

This sandbox demonstrates public vs private accessibility boundary checks:

```
package main

import "fmt"

type DBConfig struct {
    ■Host      string // Public (Capitalized)
    ■Port      int    // Public (Capitalized)
    ■password string // Private (lowercase, inaccessible outside package)
}

func NewDBConfig(host string, port int, pwd string) DBConfig {
    ■return DBConfig{
        ■■Host:      host,
        ■■Port:      port,
        ■■password:  pwd, // Accessible here because we are inside the same package
    }
}

func main() {
    ■cfg := NewDBConfig("localhost", 5432, "supersecret")
    ■
    ■fmt.Printf("Connecting to %s on port %d...\n", cfg.Host, cfg.Port)
    ■// fmt.Println(cfg.password) // UNCOMMENTING THIS LINE WILL CAUSE A COMPILE ERROR
}
```

## 3. Structs (Custom Data Types)

A **Struct** is a schema of typed fields, used to group related data.

```
type Book struct {
    Title string // Exported
    Pages int    // Exported
    author string // Unexported
}
```

## Deep Dive: Structs & JSON Tags

### In-Depth Explanation

Structs are Go's primary mechanism for representation of domain objects, schemas, request bodies, and database mappings.

- **Instantiation:** Structs can be allocated on the stack directly or as zero-valued objects: `u := User{}`.
- **Struct Tags:** Are string literals attached to fields. Go's runtime library uses reflection (`reflect` package) to read these tags. For example, during JSON decoding, the `encoding/json` package matches keys in a JSON request (like `email_address`) to the struct field tagged with `json:"email_address"`.

## Sandbox Program: Serializing Structs to JSON with Tags

This sandbox demonstrates how tags convert struct fields into customized JSON payload keys:

```
package main

import (
    "encoding/json"
    "fmt"
)

type Applicant struct {
    ID          int64 `json:"applicant_id"`
    FullName    string `json:"full_name"`
    PlainPass   string `json:"- "` // Prevents field from serialization
}

func main() {
    app := Applicant{
        ID:          1024,
        FullName:    "Aftab Khan",
        PlainPass:   "mypassword123",
    }

    // Marshal struct to JSON bytes
    bytes, err := json.MarshalIndent(app, "", " ")
    if err != nil {
        fmt.Println("Error:", err)
        return
    }

    fmt.Println("Serialized JSON Payload:")
    fmt.Println(string(bytes)) // Note that PlainPass is completely omitted
}
```

## 4. Pointers (Memory Management)

In Go, passing a variable to a function copies its value. If the struct is large, copying it wastes memory. If you modify it inside the function, the caller's copy is unaffected.

- **Pointers** store the **memory address** of a variable instead of its value.
- **Address-of Operator (&):** Prefixing a variable with `&` returns its memory address (e.g., `&user`).
- **Dereferencing Operator (asterisk):** Prefixing a pointer with the asterisk character (`*`) reads the value at that address.
- **Type Declaration (asterisk-Type):** Prefixing a type name with the asterisk character (such as `*User`) declares a pointer type that references the original struct.

## Deep Dive: Pointers & Memory Referencing

### In-Depth Explanation

Understanding Go's memory allocation model is crucial:

- 1 **Stack vs Heap:** Stack allocation is extremely fast and handled automatically. When a variable's address is shared outside the current function execution frame (escape analysis), the compiler automatically allocates it to the **Heap**.

- 2 **Nil Pointers:** Declaring a pointer without assigning it leaves it empty: `var u *User` (which is `nil`). Trying to read or write fields on a `nil` pointer triggers a runtime crash (**panic: runtime error: invalid memory address or nil pointer dereference**). Always verify pointer variables are not `nil` before usage.

## Sandbox Program: Pointers and Memory Address Referencing

This sandbox shows how pointers manipulate values at identical memory addresses:

```
package main

import "fmt"

type Profile struct {
    District string
}

// Modifies the original profile directly via pointer dereference
func changeDistrict(p *Profile, newDist string) {
    if p == nil {
        return // Avoid nil pointer panic
    }
    p.District = newDist
}

func main() {
    p := Profile{District: "Peshawar"}
    fmt.Println("Original District:", p.District)

    // Pass the memory address using & operator
    changeDistrict(&p, "Swat")
    fmt.Println("Modified District:", p.District)

    // Inspect the raw memory address
    fmt.Printf("Memory address of struct: %p\n", &p)
}
```

## Phased Struct Implementation Guide

To master structs, we will now define **every core data structure** required by the PEACE SME Grant Portal.

Create a file named `internal/db/models.go` to declare all structs.

### 1. Database Nullable Types

PostgreSQL columns can contain NULL values. Standard Go types (like `string` or `int64`) cannot hold `nil`.

- We use the standard library database/sql types: `sql.NullString`, `sql.NullInt64`, `sql.NullFloat64`, `sql.NullBool`, and `sql.NullTime`.
- To access values, inspect `.Valid` first, then access the typed field (e.g., `name.String`).

### 2. Database Entity Structs

```
package db

import (
    "database/sql"
    "encoding/json"
    "time"
)

// User models the 'users' table, storing applicant and admin authentication credentials.
type User struct {
    UserID          int64          `json:"user_id"`
    EmailAddress    string         `json:"email_address"`
    HashedPassword  string         `json:"-` // Prevents password hashes from encoding in JSON
    FirstName       sql.NullString `json:"first_name"`
    LastName        sql.NullString `json:"last_name"`
    MiddleName      sql.NullString `json:"middle_name"`
    CNIC            sql.NullString `json:"cnic"`
    Language        sql.NullString `json:"language"`
}
```

```

■Gender            sql.NullString `json:"gender"`
■MobileNo          sql.NullString `json:"mobile_no"`
■WhatsappNo        sql.NullString `json:"whatsapp_no"`
■TermsAccepted     bool           `json:"terms_accepted"`
■Status            string         `json:"status" // 'blocked' | 'unblocked'`
■LastLoginIP       sql.NullString `json:"last_login_ip"`
■DeviceFingerprint sql.NullString `json:"device_fingerprint"`
■CreatedAt          time.Time      `json:"created_at"`
}

// Business models the 'businesses' table, storing profile details.
type Business struct {
■BusinessID          int64           `json:"business_id"`
■UserID              int64           `json:"user_id"`
■NameOfBusiness      sql.NullString  `json:"name_of_business"`
■BusinessRegistrationNumber sql.NullString `json:"business_registration_number"`
■BusinessRegistrationDate sql.NullTime   `json:"business_registration_date"`
■BusinessRegistrationAuthority json.RawMessage `json:"business_registration_authority" //`
JSONB Array
■OtherAuthorityText  sql.NullString  `json:"other_authority_text"`
■BusinessFullAddress sql.NullString  `json:"business_full_address"`
■SocialMediaPage     sql.NullString  `json:"social_media_page"`
■SocialMediaPage2    sql.NullString  `json:"social_media_page_2"`
■SocialMediaPage3    sql.NullString  `json:"social_media_page_3"`
■SocialMediaPage4    sql.NullString  `json:"social_media_page_4"`
■MaleEmployees       sql.NullInt64   `json:"male_employees"`
■FemaleEmployees     sql.NullInt64   `json:"female_employees"`
■BusinessLocationDistrict sql.NullString `json:"business_location_district"`
■BusinessSector      sql.NullString  `json:"business_sector"`
■HowDidYouHear        sql.NullString  `json:"how_did_you_hear"`
■HasSRSPRelation     bool            `json:"has_srsp_relation"`
■SRSPRelativesData   json.RawMessage `json:"srsp_relatives_data" // JSONB Array`
■CreatedAt           time.Time      `json:"created_at"`
}

// BusinessDocument models uploaded files linked to businesses.
type BusinessDocument struct {
■DocumentID  int64   `json:"document_id"`
■BusinessID  int64   `json:"business_id"`
■DocumentType string  `json:"document_type"`
■FileName    string  `json:"file_name"`
■FilePath    string  `json:"file_path"`
■MIMETYPE    string  `json:"mime_type"`
■CreatedAt   time.Time `json:"created_at"`
}

// FinancedItem is a nested structure stored as JSONB in the 'grants' table.
type FinancedItem struct {
■Item      string `json:"item"`
■Quantity  int    `json:"quantity"`
■EstimatedCost float64 `json:"estimated_cost"`
}

// SRSPRelative is a nested structure stored as JSONB in the 'grants' table.
type SRSPRelative struct {
■Name      string `json:"name"`
■Position  string `json:"position"`
■Office    string `json:"office"`
}

// Grant models the complete grant application workflow.
type Grant struct {
■GrantID          int64           `json:"grant_id"`
■UserID           int64           `json:"user_id"`
■ExpressionOfInterest sql.NullString `json:"expression_of_interest" // JSON string`
array
■GrantRequired    sql.NullFloat64 `json:"grant_required"`
■ApplicationDate  sql.NullTime    `json:"application_date"`
■Status           string          `json:"status" // 'Pending', 'Approved', etc.`
■ContributionType sql.NullString  `json:"contribution_type"`
■FinancialAmount  sql.NullFloat64 `json:"financial_amount"`
■FinancialAmountWords sql.NullString `json:"financial_amount_words"`
■InKindDetails    sql.NullString  `json:"inkind_details"`
■InKindValue      sql.NullFloat64 `json:"inkind_value"`
■ContributionUtilization sql.NullString `json:"contribution_utilization"`
■GrantSupportGrowth sql.NullString  `json:"grant_support_growth"`
■JobCreationDetails sql.NullString  `json:"job_creation_details"`
}

```

```

■GrantAmountWords      sql.NullString `json:"grant_amount_words"`
■OtherPurposeText      sql.NullString `json:"other_purpose_text"`
■HowDidYouHear          sql.NullString `json:"how_did_you_hear"`
■ApprovedAmount         sql.NullFloat64 `json:"approved_amount"`
■ApprovalReason         sql.NullString `json:"approval_reason"`
■ApprovedAt            sql.NullTime `json:"approved_at"`
■ApprovedBy            sql.NullString `json:"approved_by"`
■HFCStatus              string `json:"hfc_status"`
■HFCScore               int `json:"hfc_score"`
■HFCRiskLevel           string `json:"hfc_risk_level"`
■HFCLastEvaluatedAt     sql.NullTime `json:"hfc_last_evaluated_at"`
■HFCModelVersion        sql.NullString `json:"hfc_model_version"`
■DomicileDistrict       sql.NullString `json:"domicile_district"`
■BusinessType           json.RawMessage `json:"business_type"`
■BusinessTypeOther      sql.NullString `json:"business_type_other"`
■TaxRegistrationStatus   json.RawMessage `json:"tax_registration_status"`
■NTNRegistrationNo       sql.NullString `json:"ntn_registration_no"`
■TaxFilerStatus         sql.NullString `json:"tax_filer_status"`
■WorkingCapital         bool `json:"working_capital"`
■FinancedItems          json.RawMessage `json:"financed_items"`
■ExpectedProductionIncrease sql.NullString `json:"expected_production_increase"`
■EmploymentGrid         json.RawMessage `json:"employment_grid"`
■DeclarationAccepted    bool `json:"declaration_accepted"`
■DeclarationName        sql.NullString `json:"declaration_name"`
■HasSRSPRelative        bool `json:"has_srsp_relative"`
■SRSPRelatives          json.RawMessage `json:"srsp_relatives"`
}

// GrantMedia models files associated with a grant.
type GrantMedia struct {
■MediaID   int64   `json:"media_id"`
■GrantID   int64   `json:"grant_id"`
■MediaType string   `json:"media_type"`
■FileName  string   `json:"file_name"`
■FilePath  string   `json:"file_path"`
■CreatedAt time.Time `json:"created_at"`
}

// GrantApprovalLog audits administrative approval decisions.
type GrantApprovalLog struct {
■LogID      int64   `json:"log_id"`
■UserID     int64   `json:"user_id"`
■ApprovedBy string   `json:"approved_by"`
■ApprovedAmount float64 `json:"approved_amount"`
■Reason     string   `json:"reason"`
■CreatedAt  time.Time `json:"created_at"`
}

// HFCEvaluation audits fraud scoring rule evaluations.
type HFCEvaluation struct {
■EvaluationID int64   `json:"evaluation_id"`
■UserID       int64   `json:"user_id"`
■Score        int     `json:"score"`
■RiskLevel    string   `json:"risk_level"`
■RuleDetails  string   `json:"rule_details"` // JSON string breakdown
■EvaluatedAt  time.Time `json:"evaluated_at"`
}

// HFCReviewAction models HFC clear/failed status changes by admin.
type HFCReviewAction struct {
■ActionID   int64   `json:"action_id"`
■UserID     int64   `json:"user_id"`
■ActionType string   `json:"action_type"`
■Comment    string   `json:"comment"`
■PerformedBy string   `json:"performed_by"`
■CreatedAt  time.Time `json:"created_at"`
}

// HFCRuleConfig stores weights for the deterministic scoring engine.
type HFCRuleConfig struct {
■RuleKey    string   `json:"rule_key"`
■Weight     int     `json:"weight"`
■Description string   `json:"description"`
■UpdatedAt  time.Time `json:"updated_at"`
}

// Update represents public system announcements.

```

```

type Update struct {
    UpdateID int64    `json:"update_id"`
    Title     string    `json:"title"`
    Content   string    `json:"content"`
    IsUrgent  bool      `json:"is_urgent"`
    CreatedAt time.Time `json:"created_at"`
}

// FAQ represents database elements for the FAQ bot.
type FAQ struct {
    FAQID      int64    `json:"faq_id"`
    Question   string   `json:"question"`
    Answer     string   `json:"answer"`
    Keywords   string   `json:"keywords"`
    Language   string   `json:"language"`
    CreatedAt  time.Time `json:"created_at"`
}

```

### 3. Application Configuration Structs

```

package config

// Config models application environment variables.
type Config struct {
    Port                int
    DatabaseURL         string
    RedisURL            string
    JWTSecret           string
    GrantApplicationOpen bool
    GrantRequireSelection bool
    HFCShadowMode       bool
    CachePrefix         string
    AllowedCountryCodes map[string]bool
    AdminUsers          []AdminUser
}

// AdminUser matches the schema of ADMIN_USERS_JSON configuration values.
type AdminUser struct {
    Username      string `json:"username"`
    PasswordHash  string `json:"password_hash"`
    Role          string `json:"role"`
    CanApproveGrants bool   `json:"can_approve_grants"`
}

```

### 4. Authentication Structs

```

package security

import "github.com/golang-jwt/jwt/v5"

// UserClaims models custom payload claims for JWT authentication.
type UserClaims struct {
    UserID int64 `json:"user_id"`
    jwt.RegisteredClaims
}

// AdminClaims models claims for administrative access.
type AdminClaims struct {
    Username string `json:"admin_username"`
    Role     string `json:"role"`
    IsAdmin  bool   `json:"is_admin"`
    IsApprover bool   `json:"is_approver"`
    jwt.RegisteredClaims
}

// Identity matches the context object set by auth middlewares.
type Identity struct {
    UserID      int64
    AdminUsername string
    Role        string
    IsAdmin     bool
    IsApprover  bool
}

```

### 5. DTO (Data Transfer Object) Structs

```

package dto

```

```
// LoginRequest models incoming user credential JSON bodies.
type LoginRequest struct {
    ■Email    string `json:"email"`
    ■Password string `json:"password"`
}

// RegisterRequest models applicant signup registration requests.
type RegisterRequest struct {
    ■Email    string `json:"email"`
    ■Password string `json:"password"`
    ■CNIC     string `json:"cnic"`
    ■Name     string `json:"name"`
    ■Gender   string `json:"gender"`
}

// BusinessProfileRequest represents payloads to create/update business profiles.
type BusinessProfileRequest struct {
    ■NameOfBusiness      string `json:"name_of_business"`
    ■BusinessRegistrationNumber string `json:"business_registration_number"`
    ■BusinessRegistrationAuthority []string `json:"business_registration_authority"`
    ■BusinessFullAddress  string `json:"business_full_address"`
    ■BusinessLocationDistrict string `json:"business_location_district"`
    ■BusinessSector       string `json:"business_sector"`
}

// GrantApplicationRequest models submitted grant details.
type GrantApplicationRequest struct {
    ■GrantRequired float64 `json:"grant_required"`
    ■ContributionType string `json:"contribution_type"`
    ■FinancialAmount float64 `json:"financial_amount"`
    ■InKindDetails string `json:"inkind_details"`
}
```

## Target Shape

```
cmd/server/main.go
internal/app/app.go
internal/config/config.go
internal/httpx/json.go
internal/health/handler.go
internal/db/models.go
```

main.go should do very little:

- 1 Load config.
- 2 Build the app.
- 3 Start the HTTP server.
- 4 Shut down cleanly on signals.

## Handler Style

Prefer explicit handlers:

```
func (h *Handler) Health(w http.ResponseWriter, r *http.Request) {
    httpx.WriteJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}
```

This style keeps framework magic low while learning Go.

## Mastery Check

You understand this chapter when you can:

- Explain what Go Modules are and how go.mod is used.



- Explain Go package visibility rules.
- Define a custom struct with JSON tags and explain why tag values are critical.
- Use pointers to share memory resources cleanly.
- Choose between Go value and pointer receiver models.

# Chapter 4: Configuration, Environment, and Application Toggles

## Purpose

The Flask application is controlled heavily by environment variables. The Go rewrite must preserve that behavior. In this chapter, we will study error propagation, dynamic slices, and hash maps in Go. We will then expand our configuration parser to document, parse, and validate **every application toggle and secret** required by the system.

## Foundational Concepts Explained Simply

### 1. Errors as Values

In Go, there are no exceptions (try/catch). Errors are simple values returned from functions that must be explicitly checked.

### Deep Dive: Go Errors & Wrapping

#### In-Depth Explanation

A function that can fail returns a value of type `error` (which is a built-in interface).

- **Sentinel Errors:** Package-level error constants like `sql.ErrNoRows` or custom errors declared with `errors.New()`.
- **Error Wrapping:** When returning an error, you can wrap it with context using `fmt.Errorf("failed: %w", err)`. This creates an error chain that preserves the original root error.
- **Checking Errors:** Use `errors.Is(err, target)` to check if a specific error exists in the chain, or `errors.As(err, &target)` to extract a custom error type.

### Sandbox Program: Error Wrapping and Checks

This sandbox demonstrates how to wrap errors and inspect the error chain dynamically:

```
package main

import (
    ■ "errors"
    ■ "fmt"
)

// Define sentinel error values
var ErrConfigNotFound = errors.New("configuration file not found")

func loadFile() error {
    ■ // Simulate a low-level OS read error
    ■ return ErrConfigNotFound
}

func parseConfig() error {
    ■ err := loadFile()
    ■ if err != nil {
    ■ ■ // Wrap the error with context using %w
    ■ ■ return fmt.Errorf("failed to parse environment: %w", err)
    ■ }
    ■ return nil
}

func main() {
    ■ err := parseConfig()
    ■ if err != nil {
    ■ ■ fmt.Println("Returned Error:", err)
    ■ }
}
```

```

■// Check if the ErrConfigNotFound is present in the error chain
■if errors.Is(err, ErrConfigNotFound) {
■    ■fmt.Println("Action: Falling back to default system values.")
■}
■}
}

```

## 2. Slices (Dynamic Lists)

A **Slice** is a dynamically-sized, flexible wrapper around a Go array.

- Slices grow automatically as you add items.
- You allocate them using `make()` or as slice literals: `[]int{1, 2, 3}`.

## Deep Dive: Slices, Capacity & Appending

### In-Depth Explanation

A slice contains three components:

- 1 **Pointer**: Points to the first element of the underlying array.
- 2 **Length (len)**: The number of elements currently in the slice.
- 3 **Capacity (cap)**: The maximum number of elements the slice can hold before the runtime must allocate a new, larger underlying array.

When you call `append(slice, value)`, if the length exceeds the capacity, Go automatically doubles the underlying array capacity, copies the elements, and returns the new slice pointer.

### Sandbox Program: Slice Capacity Growth

This sandbox demonstrates how Go increases a slice's capacity as elements are added:

```

package main

import "fmt"

func main() {
■// Initialize a slice of strings with length 0 and capacity 2
■names := make([]string, 0, 2)

■fmt.Printf("Initial - Len: %d, Cap: %d\n", len(names), cap(names))

■names = append(names, "Swat")
■names = append(names, "Shangla")
■fmt.Printf("After 2 Appends - Len: %d, Cap: %d\n", len(names), cap(names))

■// This third append exceeds the capacity of 2
■names = append(names, "Chitral")
■fmt.Printf("After 3 Appends (Grow) - Len: %d, Cap: %d\n", len(names), cap(names))
}

```

## 3. Maps (Key-Value Lookups)

A **Map** is a built-in hash table mapping unique keys to values.

- Declare maps using `make(map[KeyType]ValueType)`.
- Writing to a `nil` (uninitialized) map triggers a runtime crash. Always allocate maps before inserting values.

## Deep Dive: Maps & Comma-OK Checks

### In-Depth Explanation

- **Comma-OK Idiom**: In Go, looking up a missing key returns the value's zero-value (e.g. `0` for `int`, `""` for `string`). To differentiate between a key that has a zero-value vs a key that is missing, use the double assignment syntax: `val, ok := myMap[key]`.

- **Deleting Keys:** Use the built-in `delete(myMap, key)` function.

## Sandbox Program: Map Lookups and Deletions

This sandbox demonstrates lookups, checking for key existence, and deletion:

```
package main

import "fmt"

func main() {
    ■// Declare and allocate map
    ■districts := make(map[string]bool)
    ■districts["Swat"] = true
    ■districts["Shangla"] = true

    ■// Check if Swat exists
    ■if val, ok := districts["Swat"]; ok {
    ■■fmt.Printf("Swat exists! Value: %t\n", val)
    ■}

    ■// Check if Peshawar exists
    ■if _, ok := districts["Peshawar"]; !ok {
    ■■fmt.Println("Peshawar is not a supported KP district.")
    ■}

    ■// Delete a key
    ■delete(districts, "Swat")
    ■_, ok := districts["Swat"]
    ■fmt.Printf("Does Swat exist after delete? %t\n", ok)
}
```

## Detailed Environment Variables Reference

Below is a comprehensive guide to every environment variable used to control the portal's operational behavior, including validation rules, security notes, and fallback details:

### 1. HTTP Server & Security Controls

- **PORT**
- **Data Type:** Integer
- **Default / Fallback:** 8080
- **Validation Rule:** Must be a valid port number between 1 and 65535.
- **Operational Impact / Fallback:** The local TCP port the HTTP server binds to. If parsing fails, the configuration loader will return a non-nil parsing error and crash the application immediately at startup, preventing a silent boot failure on an invalid port.
- **JWT\_SECRET\_KEY**
- **Data Type:** String
- **Default / Fallback:** *Required (No Default)*
- **Validation Rule:** Must not be empty. In production, this should be a high-entropy cryptographically secure string (at least 32 characters long).
- **Operational Impact / Fallback:** Used as the HMAC symmetric key to sign and verify JSON Web Tokens (JWT) for both applicants and admins. If omitted or empty, the application will crash during startup with an error: "required environment variable JWT\_SECRET\_KEY is empty".
- **ADMIN\_USERS\_JSON**
- **Data Type:** String (JSON Array)
- **Default / Fallback:** `[]` (Empty Array)

- **Validation Rule:** Must be valid JSON array conforming to the schema: [{"username": "admin", "password\_hash": "...", "role": "admin", "can\_approve\_grants": true}].
- **Operational Impact / Fallback:** Bootstraps admin login profiles. Password hashes must be pre-hashed using the bcrypt algorithm. If the JSON is malformed, the startup sequence will abort with a deserialization error.

## 2. Database & Cache Connectivity

- POSTGRES\_HOST / POSTGRES\_PORT / POSTGRES\_USER / POSTGRES\_PASSWORD / POSTGRES\_DB
- **Data Type:** Strings / Integers
- **Defaults:** localhost / 5432 / sme\_user / sme\_password / sme\_app
- **Validation Rule:** Must be valid connectivity details.
- **Operational Impact / Fallback:** These fields are parsed and concatenated into a standard Postgres connection string (postgres://...). The Go loader will append sslmode=disable to allow local developer environments to connect without SSL certificates, but production overrides should use proper connection URLs.
- REDIS\_URL
- **Data Type:** String (Redis connection URI)
- **Default / Fallback:** redis://localhost:6379/0
- **Validation Rule:** Must be a valid URI format.
- **Operational Impact / Fallback:** Connects to the Redis cluster used for queuing background jobs (emails, risk rule audits) and storing active session keys.

## 3. Object Storage (S3-Compatible)

- S3\_ENDPOINT\_URL
- **Data Type:** String (URL)
- **Default / Fallback:** *Required*
- **Validation Rule:** Must be a valid URL with scheme (e.g. https://s3.contabostorage.com).
- **Operational Impact / Fallback:** Configures the client SDK endpoint to contact S3.
- S3\_ACCESS\_KEY / S3\_SECRET\_KEY
- **Data Type:** Strings
- **Default / Fallback:** *Required*
- **Validation Rule:** Non-empty credentials.
- **Operational Impact / Fallback:** S3 credentials. Omission results in S3 client generation errors.
- S3\_BUCKET\_NAME
- **Data Type:** String
- **Default / Fallback:** peace-economic
- **Validation Rule:** Must conform to DNS bucket naming conventions.
- **Operational Impact / Fallback:** Folder target bucket.
- S3\_PUBLIC\_BASE\_URL
- **Data Type:** String (URL)
- **Default / Fallback:** *Required*
- **Validation Rule:** Must be a valid public URL.
- **Operational Impact / Fallback:** Used by the backend to construct permanent public HTTP paths for saved business documents and media files.

## 4. Third-Party Integrations

- EMAIL\_API\_KEY
- **Data Type:** String
- **Default / Fallback:** *Required*
- **Validation Rule:** Must not be empty.
- **Operational Impact / Fallback:** Brevo API key used to deliver transactional emails to applicants when they pre-register or when a grant status changes.

## 5. Application Feature Flags & Toggles

- GRANT\_APPLICATION\_OPEN
- **Data Type:** String/Boolean (0 or 1)
- **Default / Fallback:** 0 (Closed)
- **Validation:** Checked using `getEnv( . . . ) == "1"`.
- **Operational Impact:** If set to 0, registration routes immediately return 403 Forbidden. If 1, registrations are enabled.
- GRANT\_REQUIRE\_SELECTION
- **Data Type:** String/Boolean (0 or 1)
- **Default / Fallback:** 1 (Required)
- **Operational Impact:** If 1, only whitelisted user records can proceed past initial drafts.
- HFC\_SHADOW\_MODE
- **Data Type:** String/Boolean (0 or 1)
- **Default / Fallback:** 1 (Shadow Mode Active)
- **Operational Impact:** If 1, background fraud scoring rules are executed and logged but do not block human approval workflows. If 0, rules are active blocks.
- ACCESS\_CONTROL\_ENABLED
- **Data Type:** String/Boolean (0 or 1)
- **Default / Fallback:** 1 (Active)
- **Operational Impact:** Enables concurrent session limits using Redis counters.
- MAX\_ACTIVE\_APPLICANTS
- **Data Type:** Integer
- **Default / Fallback:** 300
- **Operational Impact:** The maximum number of concurrent applicant sessions.
- GEO\_BLOCK\_ENABLED
- **Data Type:** String/Boolean (0 or 1)
- **Default / Fallback:** 1 (Active)
- **Operational Impact:** Inspects country code headers to block traffic.
- ALLOWED\_COUNTRY\_CODES
- **Data Type:** String (Comma-separated values)
- **Default / Fallback:** PK
- **Operational Impact:** Traffic outside this list is blocked. Case-insensitive and trimmed during loading.

## Phased Configuration Implementation

To master parsing, we will now define our configuration loader.

Create a file named `internal/config/config.go` to declare these structures and parser functions.

```
package config

import (
    ■"encoding/json"
    ■"fmt"
    ■"os"
    ■"strconv"
    ■"strings"
)

type AdminUser struct {
    ■Username      string `json:"username"`
    ■PasswordHash  string `json:"password_hash"`
    ■Role          string `json:"role"`
    ■CanApproveGrants bool  `json:"can_approve_grants"`
}

type Config struct {
    ■Port          int
    ■DatabaseURL   string
    ■RedisURL      string
    ■JWTSecret     string
    ■GrantApplicationOpen bool
    ■GrantRequireSelection bool
    ■HFCShadowMode bool
    ■CachePrefix   string
    ■AllowedCountryCodes map[string]bool
    ■AdminUsers     []AdminUser
}

// Load compiles and validates configuration values from the environment.
// It executes the following step-by-step loading procedure:
// 1. Os Environment Lookup: Scans environment variables on the system.
// 2. Type Conversion: Safe conversion of strings to integers (Port) and booleans.
// 3. String Interp: Concatenates PostgreSQL components into a connection URL.
// 4. Presence Validation: Verifies critical keys like JWT_SECRET_KEY exist.
// 5. Normalization: Splits allowed countries by comma and capitalizes them.
// 6. JSON Deserialization: Unmarshals ADMIN_USERS_JSON into slices.
func Load() (*Config, error) {
    ■cfg := &Config{
    ■■AllowedCountryCodes: make(map[string]bool),
    ■}

    ■// 1. Parsing integer Port
    ■portStr := getEnv("PORT", "8080")
    ■port, err := strconv.Atoi(portStr)
    ■if err != nil {
    ■■return nil, fmt.Errorf("parsing PORT %q failed: %w", portStr, err)
    ■}
    ■cfg.Port = port

    ■// 2. Database credentials compilation
    ■dbHost := getEnv("POSTGRES_HOST", "localhost")
    ■dbPort := getEnv("POSTGRES_PORT", "5432")
    ■dbUser := getEnv("POSTGRES_USER", "sme_user")
    ■dbPass := getEnv("POSTGRES_PASSWORD", "sme_password")
    ■dbName := getEnv("POSTGRES_DB", "sme_app")
    ■cfg.DatabaseURL = fmt.Sprintf("postgres://%s:%s@%s:%s/%s?sslmode=disable",
    ■■dbUser, dbPass, dbHost, dbPort, dbName)

    ■// 3. Redis URL
    ■cfg.RedisURL = getEnv("REDIS_URL", "redis://localhost:6379/0")

    ■// 4. Validate required variables
    ■cfg.JWTSecret = os.Getenv("JWT_SECRET_KEY")
    ■if cfg.JWTSecret == "" {
    ■■return nil, fmt.Errorf("required environment variable JWT_SECRET_KEY is empty")
    ■}

    ■// 5. Parsing boolean toggles
```

```

■cfg.GrantApplicationOpen = getEnv("GRANT_APPLICATION_OPEN", "0") == "1"
■cfg.GrantRequireSelection = getEnv("GRANT_REQUIRE_SELECTION", "1") == "1"
■cfg.HFCShadowMode = getEnv("HFC_SHADOW_MODE", "1") == "1"

■// 6. Split allowed country codes into a lookup map
■allowedStr := getEnv("ALLOWED_COUNTRY_CODES", "PK")
■for _, code := range strings.Split(allowedStr, ",") {
■■clean := strings.TrimSpace(strings.ToUpper(code))
■■if clean != "" {
■■■cfg.AllowedCountryCodes[clean] = true
■■}
■}

■// 7. Unmarshal admin users array
■adminJSON := os.Getenv("ADMIN_USERS_JSON")
■if adminJSON != "" {
■■var admins []AdminUser
■■if err := json.Unmarshal([]byte(adminJSON), &admins); err != nil {
■■■return nil, fmt.Errorf("decoding ADMIN_USERS_JSON failed: %w", err)
■■}
■■cfg.AdminUsers = admins
■}

■return cfg, nil
}

func getEnv(key, defaultVal string) string {
■val := os.Getenv(key)
■if val == "" {
■■return defaultVal
■}
■return val
}

```

## Practical Examples

### Deep Dive: Runnable Configuration Sandbox

#### In-Depth Explanation

This sandbox program simulates our configuration loader. It sets up a mock environment, parses integers, converts booleans, maps country codes, and validates admin JSON configs in an isolated, runnable environment.

#### Sandbox Program: Mock Environment Configuration Loader

```

package main

import (
■"encoding/json"
■"fmt"
■"os"
■"strconv"
■"strings"
)

type SandboxAdmin struct {
■Username string `json:"username"`
■Role     string `json:"role"`
}

type SandboxConfig struct {
■Port      int
■Open      bool
■AllowedCountries map[string]bool
■Admins    []SandboxAdmin
}

func main() {
■// Set mock environment variables
■os.Setenv("MOCK_PORT", "9000")
■os.Setenv("MOCK_OPEN", "1")
■os.Setenv("MOCK_COUNTRIES", "PK,AE")

```



```

■os.Setenv("MOCK_ADMINS", `[{"username":"aftab", "role":"approver"}]`)

■cfg := &SandboxConfig{
■■AllowedCountries: make(map[string]bool),
■}

■// 1. Parse Port
■portStr := os.Getenv("MOCK_PORT")
■port, err := strconv.Atoi(portStr)
■if err != nil {
■■fmt.Println("Error parsing port:", err)
■■return
■}
■cfg.Port = port

■// 2. Parse Toggle
■cfg.Open = os.Getenv("MOCK_OPEN") == "1"

■// 3. Parse Allowed Countries Map
■countriesStr := os.Getenv("MOCK_COUNTRIES")
■for _, code := range strings.Split(countriesStr, ",") {
■■cfg.AllowedCountries[strings.TrimSpace(code)] = true
■}

■// 4. Parse Admins Slice
■adminsStr := os.Getenv("MOCK_ADMINS")
■var admins []SandboxAdmin
■if err := json.Unmarshal([]byte(adminsStr), &admins); err != nil {
■■fmt.Println("Error parsing admins:", err)
■■return
■}
■cfg.Admins = admins

■// Output parsed values
■fmt.Println("--- Parsed Sandbox Config ---")
■fmt.Printf("Server Port: %d\n", cfg.Port)
■fmt.Printf("Registration Open: %t\n", cfg.Open)
■fmt.Printf("Whitelisted Countries: %v\n", cfg.AllowedCountries)
■fmt.Printf("Loaded Admins: %v\n", cfg.Admins)
}

```

## Mastery Check

You understand this chapter when you can:

- List every environmental variable that controls the portal's security state.
- Explain the role of the GRANT\_APPLICATION\_OPEN toggle.
- Parse environment variables into integers, booleans, and nested JSON structs.
- Use maps for whitelisted country lookups.
- Propagate parsing errors safely using formatting context.

# Chapter 5: PostgreSQL, Migrations, and Data Modeling

## Purpose

The database is the most important compatibility layer. The Go backend must use the same PostgreSQL schema as the Flask application. In this chapter, we will study relational modeling, database connection pools, migrations, nullable values, JSONB, and transactions. We will also learn about Go's **Generics** system and implement the generic data wrappers used across the portal API layer.

## Foundational Concepts Explained Simply

### 1. Relational Databases & Connection Pools

#### Deep Dive: Relational Databases & Connection Pools

#### In-Depth Explanation

In relational databases (like PostgreSQL), structured facts are stored in tables representing entity definitions.

- **Primary Key (PK):** A column that uniquely identifies each row (e.g. `users.id`).
- **Foreign Key (FK):** A column that references the PK of another table, creating an association.
- **One-to-One (1:1):** One user record maps to exactly one business profile (e.g., `businesses.user_id` has a unique constraint).
- **One-to-Many (1:N):** One business profile maps to multiple uploaded documents (e.g., `business_documents.business_id` references `businesses.id`).
- **Connection Pooling:** Instead of spending ~10-50 milliseconds establishing a new TCP connection and database handshake for every single incoming HTTP request, a connection pool (like Go's `pgxpool`) boots a preset amount of active connections (e.g., `MinConns=5`, `MaxConns=50`) at startup. When a query is run, the driver leases an available connection from the pool, runs the query, and instantly returns the connection to the pool.

#### Sandbox Program: Mock Connection Pool & Relational Registry

This sandbox simulates a thread-safe in-memory database registry and a connection pool using Go channels. It lets you simulate leasing connections, fetching related entities (1:N mapping), and handles concurrent leases:

```
package main

import (
    ■"context"
    ■"errors"
    ■"fmt"
    ■"time"
)

// Mock DB structures representing relational models
type User struct {
    ■ID      int64
    ■Email   string
}

type Business struct {
    ■ID      int64
    ■UserID  int64 // FK referencing User.ID
    ■Name    string
}

// Connection represents a leased database handle
```

```

type Connection struct {
    ID int
}

func (c *Connection) QueryRow(query string) string {
    return fmt.Sprintf("[Conn %d] Result for query: %s", c.ID, query)
}

// Pool maintains a channel of open connections
type Pool struct {
    conns chan *Connection
}

func NewPool(size int) *Pool {
    p := &Pool{
        conns: make(chan *Connection, size),
    }
    for i := 1; i <= size; i++ {
        p.conns <- &Connection{ID: i}
    }
    return p
}

// Acquire leases a connection or blocks until one is free or context cancels
func (p *Pool) Acquire(ctx context.Context) (*Connection, error) {
    select {
    case conn := <-p.conns:
        return conn, nil
    case <-ctx.Done():
        return nil, errors.New("context deadline exceeded waiting for connection")
    }
}

// Release returns a leased connection to the pool
func (p *Pool) Release(conn *Connection) {
    p.conns <- conn
}

func main() {
    // 1. Initialize Mock Data representing 1:1 Relational Association
    user := User{ID: 42, Email: "contact@peace.gov"}
    biz := Business{ID: 101, UserID: user.ID, Name: "Swat Organic Farms"}

    fmt.Printf("Relational Map: User %s (ID %d) owns Business %s (ID %d, FK UserID %d)\n\n",
        user.Email, user.ID, biz.Name, biz.ID, biz.UserID)

    // 2. Initialize connection pool of size 2
    pool := NewPool(2)

    // Lease 1
    ctx, cancel := context.WithTimeout(context.Background(), 100*time.Millisecond)
    defer cancel()

    conn1, err := pool.Acquire(ctx)
    if err != nil {
        fmt.Println("Acquire 1 failed:", err)
        return
    }
    fmt.Println("Acquired:", conn1.QueryRow("SELECT * FROM businesses WHERE user_id = 42"))

    // Lease 2
    conn2, err := pool.Acquire(ctx)
    if err != nil {
        fmt.Println("Acquire 2 failed:", err)
        return
    }
    fmt.Println("Acquired:", conn2.QueryRow("SELECT * FROM users WHERE id = 42"))

    // Try Lease 3 (Should fail as pool size is 2 and both are leased)
    _, err = pool.Acquire(ctx)
    if err != nil {
        fmt.Println("Acquire 3 expectedly failed:", err)
    }

    // Release one connection and try again
    pool.Release(conn1)
    fmt.Println("Released Conn 1.")
}

```

```

■conn3, err := pool.Acquire(ctx)
■if err == nil {
■■fmt.Println("Successfully acquired connection after release:", conn3.QueryRow("SELECT 1"))
■■pool.Release(conn3)
■}
■
■pool.Release(conn2)
}

```

## 2. PostgreSQL Nullable Columns

### Deep Dive: Handling SQL NULL in Go

#### In-Depth Explanation

In SQL databases, any column without a NOT NULL constraint can store a value or NULL (meaning the absence of a value). In contrast, primitive Go types (like string, int64, float64) are value types and can never hold nil.

- **The Problem:** If you attempt to scan a SQL NULL value directly into a Go primitive string variable using standard database drivers, the driver will trigger a runtime error because nil cannot be assigned to a string.
- **The Solution:** Go provides special types inside the standard database/sql package (e.g. sql.NullString, sql.NullInt64, sql.NullBool). These are structs containing:
  - 1 String / Int64 / Bool: The underlying value of that type.
  - 2 Valid: A boolean flag that is true if the database value was not null, and false if it was null.
- **JSON Considerations:** By default, serializing a sql.NullString to JSON returns an object like {"String": "value", "Valid": true}. To return a clean null (null or "value") in HTTP responses, we must implement custom JSON marshaling for these null types.

#### Sandbox Program: Serializing & Scanning Null Values

This program demonstrates how sql.NullString holds SQL null states, and how we write a wrapper type to marshal clean JSON payloads for our Vue frontend:

```

package main

import (
■"database/sql"
■"encoding/json"
■"fmt"
)

// CleanNullString wraps sql.NullString to provide clean JSON serialization
type CleanNullString struct {
■sql.NullString
}

// MarshalJSON override converts the struct into clean JSON
func (c CleanNullString) MarshalJSON() ([]byte, error) {
■if !c.Valid {
■■return json.Marshal(nil)
■}
■return json.Marshal(c.String)
}

// UnmarshalJSON override lets us decode nulls back into our struct
func (c *CleanNullString) UnmarshalJSON(data []byte) error {
■if string(data) == "null" {
■■c.Valid = false
■■c.String = ""
■■return nil
■}
■var s string
■if err := json.Unmarshal(data, &s); err != nil {
■■return err
■}
■c.String = s
■c.Valid = true
}

```

```

return nil
}

type GrantApplication struct {
    ID int64 `json:"id"`
    ApprovedAmount float64 `json:"approved_amount"`
    ApprovalReason CleanNullString `json:"approval_reason" // Nullable in DB
}

func main() {
    // 1. Simulating an application that has NOT yet been approved (Reason is NULL)
    app1 := GrantApplication{
        ID: 1,
        ApprovedAmount: 0.0,
        ApprovalReason: CleanNullString{sql.NullString{String: "", Valid: false}},
    }

    // 2. Simulating an approved application (Reason is populated)
    app2 := GrantApplication{
        ID: 2,
        ApprovedAmount: 150000.0,
        ApprovalReason: CleanNullString{sql.NullString{String: "Approved by HFC board", Valid: true}},
    }

    // Marshal both to JSON
    json1, _ := json.Marshal(app1)
    json2, _ := json.Marshal(app2)

    fmt.Println("JSON output for App 1 (NULL reason):", string(json1))
    fmt.Println("JSON output for App 2 (Valid reason):", string(json2))

    // Decode back
    var decodedApp GrantApplication
    _ = json.Unmarshal(json1, &decodedApp)
    fmt.Printf("Decoded App 1 -> Valid: %t, Value: %q\n",
        decodedApp.ApprovalReason.Valid, decodedApp.ApprovalReason.String)
}

```

### 3. Generics in Go (Type Parameters)

#### Deep Dive: Go Generics & Type Parameters

##### In-Depth Explanation

Before Go 1.18, writing reusable data structures (like lists, maps, or unified API envelopes) required using the empty interface `interface{}` (which is now alias-named `any`).

- **The Problem with Interface-based Reusability:** Operating on `interface{}` values requires manual runtime type assertions (e.g. `val.(concreteType)`). If you perform an incorrect type assertion, the program crashes at runtime. This bypasses compile-time type safety.
- **The Generics Solution:** Generics allow you to parameterize structs, functions, or methods with type parameters (declared in square brackets like `[T any]`).
- `T` is a placeholder that is substituted with a concrete type when the struct is initialized or the function is called.
- **Type Constraints:** You can limit what types can fill the placeholder. The constraint `any` allows any type. The constraint `comparable` restricts to types that support comparison operators (`==` and `!=`).

#### Sandbox Program: Type-Safe Generic Cache Registry

This sandbox demonstrates how to create a generic Cache container that stores any data type `T` type-safely, preventing runtime errors by enforcing strict types at compile time:

```

package main

import (
    "fmt"
)

// CacheEntry is a generic container holding type T

```

```

type CacheEntry[T any] struct {
    ■Key    string
    ■Value  T
}

// KeyValuePair registry using generics
type CacheRegistry[T any] struct {
    ■store map[string]CacheEntry[T]
}

func NewCacheRegistry[T any]() *CacheRegistry[T] {
    ■return &CacheRegistry[T]{
    ■■store: make(map[string]CacheEntry[T]),
    ■}
}

func (cr *CacheRegistry[T]) Set(key string, val T) {
    ■cr.store[key] = CacheEntry[T]{Key: key, Value: val}
}

func (cr *CacheRegistry[T]) Get(key string) (T, bool) {
    ■entry, exists := cr.store[key]
    ■if !exists {
    ■■var zero T // Returns the zero value of type T
    ■■return zero, false
    ■}
    ■return entry.Value, true
}

func main() {
    ■// 1. Create a cache registry specifically for integer IDs
    ■intCache := NewCacheRegistry[int]()
    ■intCache.Set("active_session_count", 245)
    ■
    ■val, _ := intCache.Get("active_session_count")
    ■fmt.Printf("Int Cache - Value: %d (Type: %T)\n", val, val)

    ■// 2. Create a cache registry specifically for string user roles
    ■stringCache := NewCacheRegistry[string]()
    ■stringCache.Set("user_42_role", "admin")
    ■
    ■role, _ := stringCache.Get("user_42_role")
    ■fmt.Printf("String Cache - Value: %s (Type: %T)\n", role, role)

    ■// Note: Trying to do stringCache.Set("id", 123) would fail at compile time!
}

```

## External Resources

- [Go Dev: Tutorial on Generics](#)
- [pgx Connection Pool Documentation](#)
- [System Design Primer: Database Connection Pools](#)

## Phased Generics Implementation Guide

To practice generics, we will now define **all generic wrapper structures** used by the PEACE SME Grant Portal's HTTP layers.

Create a file named `internal/httpx/json.go` to declare these structures.

### 1. Generic API Response Wrapper

Every API response returned to the Vue frontend should share a consistent envelope layout containing success status, metadata messages, and payload data:

```

package httpx

import (
    ■"encoding/json"
    ■"net/http"
)

```

```
// APIResponse wraps any payload type T in a standard HTTP envelope.
type APIResponse[T any] struct {
    ■Success bool    `json:"success"`
    ■Message string `json:"message,omitempty"`
    ■Data    T        `json:"data,omitempty"`
}

// WriteJSON writes a type-safe generic response.
func WriteJSON[T any](w http.ResponseWriter, status int, data T) {
    ■w.Header().Set("Content-Type", "application/json")
    ■w.WriteHeader(status)

    ■envelope := APIResponse[T]{
        ■■Success: status < 400,
        ■■Data:    data,
        ■}

    ■_ = json.NewEncoder(w).Encode(envelope)
}
```

## 2. Generic Paginated Response Container

For administrative tables and audit report views, pages must return paginated list data along with paging counts:

```
// PaginatedData holds list items and pagination metadata.
type PaginatedData[T any] struct {
    ■Items []T    `json:"data"`
    ■Total int64 `json:"total"`
    ■Page  int    `json:"page"`
    ■PerPage int   `json:"per_page"`
}

// WritePaginatedJSON renders paginated lists type-safely.
func WritePaginatedJSON[T any](w http.ResponseWriter, status int, items []T, total int64, page
int, perPage int) {
    ■payload := PaginatedData[T]{
        ■■Items: items,
        ■■Total: total,
        ■■Page:  page,
        ■■PerPage: perPage,
        ■}
    ■WriteJSON(w, status, payload)
}
```

## Tables to Preserve

The core tables are:

- users
- businesses
- business\_documents
- grants
- grant\_media
- grant\_approval\_logs
- grant\_access\_whitelist
- applicant\_status
- hfc\_evaluations
- hfc\_review\_actions
- hfc\_rule\_config
- initial\_registrations
- updates

- [faqs](#)
- [schema\\_migrations](#)

## Practical Examples

### Example: Executing a PostgreSQL Database Transaction in Go

This repository transaction updates a grant status and creates an approval log atomically:

```
// File: internal/grant/repository.go
package grant

import (
    ■"context"
    ■"fmt"
    ■"time"

    ■"github.com/jackc/pgx/v5/pgxpool"
)

type Repository struct {
    ■db *pgxpool.Pool
}

func (r *Repository) ApproveGrant(ctx context.Context, userID int64, approver string, amount float64, reason string) error {
    ■tx, err := r.db.Begin(ctx)
    ■if err != nil {
    ■■return fmt.Errorf("failed to begin transaction: %w", err)
    ■}
    ■// Defer a rollback; if tx commits successfully first, rollback is a safe no-op.
    ■defer tx.Rollback(ctx)

    ■updateQuery := `
    ■■UPDATE grants
    ■■SET status = 'Approved', approved_amount = $1, approval_reason = $2, approved_at = $3,
    ■■approved_by = $4
    ■■WHERE user_id = $5`
    ■
    ■result, err := tx.Exec(ctx, updateQuery, amount, reason, time.Now(), approver, userID)
    ■if err != nil {
    ■■return fmt.Errorf("failed to update grant: %w", err)
    ■}
    ■if result.RowsAffected() == 0 {
    ■■return fmt.Errorf("grant record not found for user %d", userID)
    ■}

    ■logQuery := `
    ■■INSERT INTO grant_approval_logs (user_id, approved_by, approved_amount, reason, created_at)
    ■■VALUES ($1, $2, $3, $4, $5)`
    ■
    ■_, err = tx.Exec(ctx, logQuery, userID, approver, amount, reason, time.Now())
    ■if err != nil {
    ■■return fmt.Errorf("failed to write audit log: %w", err)
    ■}

    ■if err := tx.Commit(ctx); err != nil {
    ■■return fmt.Errorf("failed to commit transaction: %w", err)
    ■}

    ■return nil
}
```

## Mastery Check

You understand this chapter when you can:

- Explain what type parameters [T any] solve in Go.
- Declare and initialize a generic structure type-safely.



- Explain why a connection pool handles concurrent HTTP request volume better than raw single connections.
- Write a database transaction block checking row-affected results and utilizing deferred rollbacks.

# Chapter 6: Authentication, JWT, bcrypt, and Middleware

## Purpose

Authentication is where backend compatibility matters immediately. The Vue frontend stores tokens and sends them as opaque bearer strings. The Go backend must issue and verify compatible JWTs. In this chapter, we will learn how Go handles functions as first-class citizens, how to use closures to configure handlers, and how `context.Context` propagates metadata. We will then build an authentication middleware chain.

Application parallel: this chapter builds the security boundary for both applicants and admins. The applicant uses `UserLogin.vue` and receives `userToken`. The admin uses `AdminLogin.vue` and receives `adminToken`. The same Go server must understand both tokens, but permissions are different.

## Foundational Concepts Explained Simply

### 1. First-Class Functions & Closures

#### Deep Dive: First-Class Functions & Closures

#### In-Depth Explanation

In Go, functions are **first-class citizens**, meaning they are treated like any other value type:

- They can be assigned to variables.
- They can be passed as arguments to other functions.
- They can be returned as output values from other functions.
- **Closures:** A closure is an anonymous function that references (captures) variables from outside its immediate scope. The function "closes over" these variables, keeping them alive and carrying them around as long as the closure exists.
- **Middleware Signature:** In Go, HTTP middleware uses closures to wrap handlers: `func(http.Handler) http.Handler`. The outer function accepts a dependency (like a configuration or database helper) and returns a closure that wraps the subsequent handler in the pipeline.

#### Sandbox Program: Middleware Logger Closure

This sandbox demonstrates how to use closures to build an HTTP middleware decorator that injects logging capabilities around a standard handler function:

```
package main

import (
    ■ "fmt"
    ■ "time"
)

// MockResponseWriter simulates http.ResponseWriter
type MockResponseWriter struct {
    ■ Status int
}

func (m *MockResponseWriter) WriteHeader(status int) {
    ■ m.Status = status
}

// HandlerFunc type matching net/http
type HandlerFunc func(w *MockResponseWriter, path string)

// LoggerMiddleware wraps a handler func inside a closure, injecting log context
```

```

func LoggerMiddleware(environment string, next HandlerFunc) HandlerFunc {
    return func(w *MockResponseWriter, path string) {
        start := time.Now()
        fmt.Printf("[%s] Incoming Request: %s\n", environment, path)

        // Execute the wrapped handler
        next(w, path)

        fmt.Printf("[%s] Completed Request in %s. Status: %d\n",
            environment, time.Since(start), w.Status)
    }
}

func main() {
    // A simple mock HTTP handler
    welcomeHandler := func(w *MockResponseWriter, path string) {
        time.Sleep(10 * time.Millisecond) // Simulate work
        w.WriteHeader(200)
        fmt.Println(" -> Executed Welcome Handler: Hello from PEACE SME!")
    }

    // Wrap the handler with our closure-based middleware
    devPipeline := LoggerMiddleware("DEVELOPMENT", welcomeHandler)

    w := &MockResponseWriter{}
    devPipeline(w, "/api/dashboard")
}

```

## 2. Context (context.Context)

### Deep Dive: Go Context & Request-Scoped Values

#### In-Depth Explanation

When a web request is processed by a Go server, the request traverses a pipeline of middleware and handlers.

- **Request Lifecycle:** Go's `context.Context` propagates deadlines, cancellation signals, and request-scoped values down the call stack.
- **Injecting Values:** Use `context.WithValue(parent, key, value)`. Because contexts are immutable, this returns a *new* child context carrying the key-value pair.
- **Type-Safe Key Rule:** Always declare a custom, unexported type for context keys (e.g. `type contextKey struct{}`) rather than using raw strings. This prevents other packages from accidentally overwriting your context values.
- **Extraction:** Use `ctx.Value(key)` and convert the returned `interface{}` to the concrete target type using a type assertion `v.(Type)`.

#### Sandbox Program: Type-Safe Request Context Propagation

This sandbox simulates request context propagation. It injects a user identity structure, passes the context down, and safely retrieves it using unexported keys:

```

package main

import (
    "context"
    "fmt"
)

// 1. Declare unexported custom types for context keys to prevent collisions
type contextKey struct{}

var userKey = contextKey{}

type UserIdentity struct {
    UserID int64
    Role   string
}

// InjectUser returns a new context containing the identity

```

```

func InjectUser(ctx context.Context, id int64, role string) context.Context {
    ■identity := UserIdentity{UserID: id, Role: role}
    ■return ctx.WithValue(ctx, userKey, identity)
}

// ExtractUser retrieves the identity from context type-safely
func ExtractUser(ctx context.Context) (UserIdentity, bool) {
    ■val, ok := ctx.Value(userKey).(UserIdentity)
    ■return val, ok
}

func main() {
    ■// Start with a root background context
    ■rootCtx := context.Background()

    ■// Inject identity (simulating authentication middleware)
    ■reqCtx := InjectUser(rootCtx, 42, "applicant")

    ■// Call repository (simulating database query layer)
    ■fetchData(reqCtx)
}

func fetchData(ctx context.Context) {
    ■// Retrieve values from context
    ■identity, ok := ExtractUser(ctx)
    ■if !ok {
    ■■fmt.Println("Error: No identity found in request context!")
    ■■return
    ■}

    ■fmt.Printf("Database Query: Executing search scoped to User ID: %d (Role: %s)\n",
    ■■identity.UserID, identity.Role)
}

```

### 3. JSON Web Tokens (JWT)

#### Deep Dive: JSON Web Tokens (JWT) Signing & Verification

##### In-Depth Explanation

A JSON Web Token (JWT) is a compact, URL-safe container used to transmit information as a JSON object.

- **Structure:** Consists of three parts separated by dots (.): Header (signing algorithm), Payload (the claim details like user\_id and expiry exp), and Signature.
- **HMAC Signing (HS256):** The signature is created by hashing the base64-encoded Header and Payload together with a secret key using HMAC-SHA256.
- **Stateless Auth:** The server does not store active tokens in a database. When a request arrives, the server recalculates the signature using its secret key. If the calculated signature matches the token's signature and the expiry exp time is in the future, the token is verified.

##### Sandbox Program: Mock HMAC Token Signature Verification

This program implements base64-encoded signing and signature verification using Go's built-in crypto/hmac package. It demonstrates how tampering with the token payload invalidates the signature:

```

package main

import (
    ■"crypto/hmac"
    ■"crypto/sha256"
    ■"encoding/base64"
    ■"fmt"
    ■"strings"
)

// Sign generates a mock JWT structure: header.payload.signature
func Sign(payload string, secret []byte) string {
    ■header := "HS256"
    ■h64 := base64.RawURLEncoding.EncodeToString([]byte(header))
    ■p64 := base64.RawURLEncoding.EncodeToString([]byte(payload))

```

```

■unsignedToken := h64 + "." + p64
■
■// Create HMAC signature
■h := hmac.New(sha256.New, secret)
■h.Write([]byte(unsignedToken))
■sig := base64.RawURLEncoding.EncodeToString(h.Sum(nil))
■
■return unsignedToken + "." + sig
}

// Verify validates that the signature matches and has not been altered
func Verify(token string, secret []byte) (string, error) {
■parts := strings.Split(token, ".")
■if len(parts) != 3 {
■return "", fmt.Errorf("invalid token format")
■}

■unsignedToken := parts[0] + "." + parts[1]
■providedSig := parts[2]

■// Recalculate HMAC signature
■h := hmac.New(sha256.New, secret)
■h.Write([]byte(unsignedToken))
■expectedSig := base64.RawURLEncoding.EncodeToString(h.Sum(nil))

■// Compare signatures in constant time to prevent timing attacks
■if !hmac.Equal([]byte(providedSig), []byte(expectedSig)) {
■return "", fmt.Errorf("signature verification failed (tampered token)")
■}

■// Decode payload
■payloadBytes, _ := base64.RawURLEncoding.DecodeString(parts[1])
■return string(payloadBytes), nil
}

func main() {
■secret := []byte("secret-portal-key-12345")
■payload := `{"user_id":42,"role":"applicant"}`

■// 1. Generate token
■token := Sign(payload, secret)
■fmt.Println("Generated Token:", token)

■// 2. Verify valid token
■decoded, err := Verify(token, secret)
■if err == nil {
■fmt.Println("Verification Success! Payload:", decoded)
■}

■// 3. Simulate tampering (change user_id from 42 to 1)
■tamperedParts := strings.Split(token, ".")
■tamperedPayload := base64.RawURLEncoding.EncodeToString([]byte(`{"user_id":1,"role":
"applicant"}`))
■tamperedToken := tamperedParts[0] + "." + tamperedPayload + "." + tamperedParts[2]
■fmt.Println("\nTampered Token:", tamperedToken)

■_, err = Verify(tamperedToken, secret)
■if err != nil {
■fmt.Println("Verification Failed as expected:", err)
■}
}

```

## 4. bcrypt Password Hashing

### Deep Dive: Bcrypt Hashing & Password Safety

#### In-Depth Explanation

- **Why Plain Hashing (SHA256) is Weak:** Simple cryptographic hash functions (like SHA256) are designed to be fast. An attacker who steals a database of SHA256 hashes can run billions of password guesses per second (using GPU-accelerated rainbow tables) to crack them.
- **The Bcrypt Solution:** Bcrypt is a key-derivation function that incorporates:

- 1 **A Salt:** A random value generated automatically for each password. This ensures two users with the same password will have completely different hashes.
  - 2 **Work Factor (Cost):** An exponent (e.g. 10 to 14) that controls how many iteration rounds the algorithm executes. This makes the hashing process intentionally slow (e.g. taking 100-300 milliseconds per hash). This slowness completely defeats brute-force attempts while remaining unnoticeable to individual logging-in users.
- **Verification:** Since bcrypt hashes are salted, you can never "decrypt" the hash. Instead, you pass the candidate password and the stored hash into a comparison library. The library extracts the salt from the hash, hashes the candidate password with that salt, and compares the results.

## Sandbox Program: Bcrypt Simulation & Password Checks

This program simulates how bcrypt uses salts to create unique hashes for identical inputs and verify password attempts securely:

```
package main

import (
    ■"crypto/sha256"
    ■"encoding/hex"
    ■"fmt"
    ■"math/rand"
)

// SimulatedBcryptHash creates a mock salted hash simulating bcrypt behaviour
func SimulatedBcryptHash(password string, cost int) (string, string) {
    ■// Generate a mock random salt
    ■saltBytes := make([]byte, 8)
    ■rand.Read(saltBytes)
    ■salt := hex.EncodeToString(saltBytes)

    ■// Hash password + salt multiple times (cost)
    ■hash := password + salt
    ■for i := 0; i < cost*1000; i++ {
    ■■h := sha256.Sum256([]byte(hash))
    ■■hash = hex.EncodeToString(h[:])
    ■}
    ■return hash, salt
}

// SimulatedBcryptVerify verifies if the candidate matches
func SimulatedBcryptVerify(password, storedHash, salt string, cost int) bool {
    ■hash := password + salt
    ■for i := 0; i < cost*1000; i++ {
    ■■h := sha256.Sum256([]byte(hash))
    ■■hash = hex.EncodeToString(h[:])
    ■}
    ■return hash == storedHash
}

func main() {
    ■password := "SecurePass123"
    ■cost := 5 // Simulates work factor iterations

    ■// 1. Hash password twice (demonstrates different salts yield different hashes)
    ■hash1, salt1 := SimulatedBcryptHash(password, cost)
    ■hash2, salt2 := SimulatedBcryptHash(password, cost)

    ■fmt.Println("Password:", password)
    ■fmt.Printf("Hash 1: %s (Salt: %s)\n", hash1[:32], salt1)
    ■fmt.Printf("Hash 2: %s (Salt: %s)\n\n", hash2[:32], salt2)

    ■// 2. Verify correct attempt
    ■ok := SimulatedBcryptVerify("SecurePass123", hash1, salt1, cost)
    ■fmt.Println("Verify with correct password:", ok)

    ■// 3. Verify incorrect attempt
    ■bad := SimulatedBcryptVerify("WrongPass", hash1, salt1, cost)
    ■fmt.Println("Verify with wrong password:", bad)
}
```

## External Resources

- Go Tour: Function Closures
- Go Blog: Context Package Explained
- Go Web Examples: Middleware

## User JWT

Payload:

```
{ "user_id": 42, "exp": 1234567890 }
```

Rules:

- HS256.
- Secret from JWT\_SECRET\_KEY.
- Expiry: 24 hours.

Beginner explanation: a JWT is a signed string. The server does not need to store it in a sessions table. The browser sends it back, and the server verifies the signature. If someone edits the `user_id` inside the token, verification fails.

## Admin JWT

Payload:

```
{
  "admin_username": "admin1",
  "role": "admin",
  "is_admin": true,
  "is_approver": false,
  "exp": 1234567890
}
```

Rules:

- HS256.
- Secret from JWT\_SECRET\_KEY.
- Expiry: 8 hours.
- `is_approver` comes from `can_approve_grants`.

Application parallel: loading an admin report requires admin identity. Approving a grant requires approver identity. That is why `is_admin` and `is_approver` are separate claims.

## bcrypt

Use [golang.org/x/crypto/bcrypt](https://golang.org/x/crypto/bcrypt):

```
err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
```

Never log plaintext passwords. Never return whether the email or password was wrong separately for normal user login.

Beginner explanation: bcrypt is intentionally slow. You never decrypt a bcrypt hash. You compare a password attempt against the stored hash. This protects users if hashes are ever exposed.

## Middleware Pipeline

The original request pipeline is:

```
apply_geo_block()
apply_access_control()
authenticate_token()
```

In Go, build composable middleware:

```
handler := Recover(
  Logger(
    GeoBlock(
      AccessControl(
        Auth(router),
      ),
    ),
  ),
)
```

Admin routes skip geo-block. Public routes skip auth. Applicant-only and admin-only routes require different token claims.

## Application Parallel: Route Categories

| Route category | Example                                  | Middleware needed                                       | Identity needed      |
|----------------|------------------------------------------|---------------------------------------------------------|----------------------|
| Public         | GET /api/updates                         | logging, recovery                                       | none                 |
| Public auth    | POST /api/login                          | logging, recovery, applicant access controls if enabled | none                 |
| Applicant      | GET /api/business                        | user auth                                               | user_id              |
| Admin          | GET /api/admin/applicants/report         | admin auth                                              | admin_username, role |
| Approver       | POST /api/admin/grants/<user_id>/approve | admin auth plus approver check                          | is_approver=true     |

The beginner mistake is to make one global middleware decision for every route. This portal has public pages, applicant pages, admin pages, and approver-only actions.

## Context Values

After auth, attach identity to request context:

```
type Identity struct {
  UserID      int64
  AdminUsername string
  Role        string
  IsAdmin      bool
  IsApprover   bool
}
```

Handlers can read identity from context without reparsing JWTs.

Use a helper instead of type assertions everywhere:

```
func IdentityFromContext(ctx context.Context) (Identity, bool) {
  identity, ok := ctx.Value(identityKey{}).(Identity)
  return identity, ok
}
```

Application parallel: `BusinessHandler.Get` reads `user_id` from context and returns only that user's business. It should never accept `user_id` from the applicant request body for applicant-owned data.

## Practical Examples

### Example 1: JWT Verification Token Service



This helper generates and parses claims using `github.com/golang-jwt/jwt/v5`:

```
// File: internal/security/jwt.go
package security

import (
    "errors"
    "fmt"
    "time"

    "github.com/golang-jwt/jwt/v5"
)

type UserClaims struct {
    UserID int64 `json:"user_id"`
    jwt.RegisteredClaims
}

type JWTService struct {
    secretKey []byte
}

func NewJWTService(secret string) *JWTService {
    return &JWTService{secretKey: []byte(secret)}
}

// GenerateUserToken generates a 24-hour token for applicants.
func (s *JWTService) GenerateUserToken(userID int64) (string, error) {
    claims := UserClaims{
        UserID: userID,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
            IssuedAt:  jwt.NewNumericDate(time.Now()),
        },
    }

    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(s.secretKey)
}

// VerifyUserToken decodes and validates a user token.
func (s *JWTService) VerifyUserToken(tokenStr string) (int64, error) {
    token, err := jwt.ParseWithClaims(tokenStr, &UserClaims{}, func(t *jwt.Token) (interface{}, error) {
        if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, fmt.Errorf("unexpected signing method: %v", t.Header["alg"])
        }
        return s.secretKey, nil
    })

    if err != nil {
        return 0, err
    }

    claims, ok := token.Claims.(*UserClaims)
    if !ok || !token.Valid {
        return 0, errors.New("invalid token claims")
    }

    return claims.UserID, nil
}
```

## Example 2: Closure-based Middleware injecting Context Identity

This middleware uses a closure pattern to receive dependencies (`JWTService`) and returns an `http.Handler` wrapper that injects validated identities into `r.Context()`:

```
// File: internal/security/middleware.go
package security

import (
    "context"
    "net/http"
    "strings"
)

type contextKey string
```

```

const UserIdentityKey contextKey = "userIdentity"

type Identity struct {
    ■UserID int64
}

// AuthMiddleware is a closure wrapping handlers with authorization checks.
func AuthMiddleware(jwtSvc *JWTService) func(http.Handler) http.Handler {
    ■return func(next http.Handler) http.Handler {
        ■■return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            ■■■authHeader := r.Header.Get("Authorization")
            ■■■if authHeader == "" {
                ■■■■http.Error(w, `{"error":"missing authorization header"}`, http.StatusUnauthorized)
                ■■■■return
            }

            ■■■// Expecting "Bearer <token>"
            ■■■parts := strings.Split(authHeader, " ")
            ■■■if len(parts) != 2 || strings.ToLower(parts[0]) != "bearer" {
                ■■■■http.Error(w, `{"error":"invalid authorization format"}`, http.StatusUnauthorized)
                ■■■■return
            }

            ■■■userID, err := jwtSvc.VerifyUserToken(parts[1])
            ■■■if err != nil {
                ■■■■http.Error(w, `{"error":"invalid or expired token"}`, http.StatusUnauthorized)
                ■■■■return
            }

            ■■■// Injected Identity into Request Context
            ■■■ctx := context.WithValue(r.Context(), UserIdentityKey, &Identity{UserID: userID})
            ■■■
            ■■■// Call next handler in decorator chain
            ■■■next.ServeHTTP(w, r.WithContext(ctx))
            ■■■})
        }
    }

    // GetIdentity extracts the Identity object from the context.
    func GetIdentity(ctx context.Context) (*Identity, bool) {
        ■id, ok := ctx.Value(UserIdentityKey).(*Identity)
        ■return id, ok
    }
}

```

## Security Rules

Preserve:

- Blocked users cannot login.
- Applicant status does not block login.
- Admin approval endpoint requires approver identity.
- CSV export query tokens are short-lived and separate from normal bearer auth.
- Geo-block checks CF-IPCountry first, then X-Country-Code.

## Mastery Check

You understand this chapter when you can:

- Explain how functions behave as values in Go.
- Write a simple closure that captures outside variables.
- Write a custom middleware function with the `func(http.Handler) http.Handler` signature.
- Propagate values securely through request scopes using `context.WithValue()`.

# Chapter 7: Applicant Registration, Login, and Business Profiles

## Purpose

This chapter implements the applicant foundation: closed registration behavior, login, profile lookup, and business profile create/update. In this chapter, we will study **Method Receivers**, learning the difference between attaching behaviors to structs by value vs by pointer. We will then implement struct validation receiver methods for our Request DTOs.

For a beginner Go developer, this is the first real vertical slice. You will move from HTTP JSON to service rules to SQL and back to Vue form state.

## Theory: Vertical Slices

A vertical slice is a small feature implemented through every layer:

```
Vue form -> Axios request -> Go handler -> service -> repository -> PostgreSQL -> JSON
response -> Vue state
```

Building one complete slice teaches more than building ten disconnected helpers. Login and business profile are ideal first slices because they touch authentication, validation, database queries, and frontend state without requiring the full grant workflow yet.

## Foundational Concepts Explained Simply

### Method Receivers (Value vs. Pointer Receivers)

### Deep Dive: Method Receivers: Value vs. Pointer

### In-Depth Explanation

Go does not support object-oriented classes, but it allows you to define behaviors (methods) on struct types. A method is declared like a regular function, but it includes an extra **Receiver** parameter located between the `func` keyword and the function name.

- **Value Receiver:** Declared as `func (s StructType) MethodName()`.
- **Mechanism:** When the method is invoked, Go creates a shallow copy of the entire struct in memory.
- **Mutability:** Any changes made to the struct's fields inside the method only affect the copy, leaving the original struct unmodified.
- **When to Use:** Use value receivers for small, read-only structures where copying is cheap, or when you want to guarantee immutability.
- **Pointer Receiver:** Declared as `func (s *StructType) MethodName()`.
- **Mechanism:** Go passes the memory address of the struct. No copying of fields occurs.
- **Mutability:** Any changes made to the struct's fields inside the method directly alter the original instance.
- **When to Use:** Use pointer receivers if the method needs to modify struct values, or if the struct contains many fields (making copying expensive).
- **Consistency Rule:** If any method on a struct requires a pointer receiver, all other methods on that struct should also use pointer receivers for consistency.

### Sandbox Program: Struct Mutation and Validation Checks

This program demonstrates how a value receiver fails to modify a struct's state while a pointer receiver successfully mutates it. It also shows a custom validation and normalization routine:

```
package main

import (
    ■ "fmt"
    ■ "strings"
)

type BusinessProfile struct {
    ■ Name      string
    ■ District string
}

// Value Receiver: Attempts to capitalize the name but works on a copy
func (b BusinessProfile) NormaliseNameValue() {
    ■ b.Name = strings.ToUpper(b.Name)
    ■ fmt.Printf("[Inside Value Receiver] Normalised Name: %s\n", b.Name)
}

// Pointer Receiver: Successfully capitalises the name in place
func (b *BusinessProfile) NormaliseNamePointer() {
    ■ b.Name = strings.ToUpper(b.Name)
    ■ fmt.Printf("[Inside Pointer Receiver] Normalised Name: %s\n", b.Name)
}

func main() {
    ■ // 1. Initialise struct
    ■ profile := BusinessProfile{
    ■     ■ Name:      "Swat Honey Extractors",
    ■     ■ District: "Swat",
    ■ }
    ■ fmt.Println("Initial Profile Name:", profile.Name)

    ■ // 2. Call Value Receiver
    ■ profile.NormaliseNameValue()
    ■ fmt.Println("Profile Name after Value Receiver call:", profile.Name)
    ■ // Output: Still "Swat Honey Extractors" because it worked on a copy!

    ■ fmt.Println()

    ■ // 3. Call Pointer Receiver
    ■ profile.NormaliseNamePointer()
    ■ fmt.Println("Profile Name after Pointer Receiver call:", profile.Name)
    ■ // Output: "SWAT HONEY EXTRACTORS" because it modified the original!
}
```

## External Resources

- [A Tour of Go: Methods](#)
- [Go Dev: FAQ on Receiver Types](#)

## Endpoints

Public:

| Method | Path                  |
|--------|-----------------------|
| POST   | /api/pre-registration |
| POST   | /api/register         |
| POST   | /api/login            |
| POST   | /api/forgot-password  |
| POST   | /api/reset-password   |

User:

| Method | Path              |
|--------|-------------------|
| GET    | /api/user/profile |
| GET    | /api/business     |
| POST   | /api/business     |
| PUT    | /api/business     |

## Registration Toggle

The current operational state has registration closed. Preserve:

- /api/pre-registration returns 403 when GRANT\_APPLICATION\_OPEN=0.
- /api/register returns 403 when GRANT\_APPLICATION\_OPEN=0.
- Password reset endpoints return 403 because the feature is disabled.

This is a good lesson: compatibility sometimes means reproducing disabled behavior.

Application parallel: the frontend has registration screens, but production config currently closes registration. The Go backend must enforce this even if a user manually calls the API.

Go concept: configuration-driven branching.

```
if !cfg.GrantApplicationOpen {
    return ErrRegistrationClosed
}
```

Vue concept: show the closed-registration UI when the API returns 403 or when a public status endpoint says registration is closed.

## Business Profile Rules

The app allows one business per user:

```
businesses.user_id INTEGER UNIQUE NOT NULL
```

Allowed districts:

- Swat
- Shangla
- Upper Dir
- Upper Chitral
- Lower Chitral

Reject any other district in create/update logic.

Go concept: validate before write.

```
func ValidateDistrict(district string) error {
    if !AllowedDistricts[district] {
        return fmt.Errorf("district %q is not allowed", district)
    }
    return nil
}
```

Database concept: the unique constraint protects data even if two requests arrive at the same time. Application validation improves the error message, but the database is the final guard.

# Request Validation

Validate at the boundary:

- Required fields.
- Email shape.
- CNIC as string.
- Boolean fields.
- JSON arrays such as `business_registration_authority`.
- District allow-list.

Do not trust the frontend. Vue validation improves UX; Go validation protects the system.

## Practical Examples

### Example 1: Defining a Request DTO with Pointer Receiver Validation

This example declares a data structure for business profiles and implements a validation method using a pointer receiver to optimize performance:

```
// File: internal/business/dto.go
package business

import (
    ■ "errors"
    ■ "strings"
)

var allowedDistricts = map[string]bool{
    ■ "swat":          true,
    ■ "shangla":       true,
    ■ "upper dir":     true,
    ■ "upper chitral": true,
    ■ "lower chitral": true,
}

type CreateBusinessRequest struct {
    ■ Name          string `json:"name_of_business"`
    ■ Address       string `json:"business_full_address"`
    ■ District      string `json:"business_location_district"`
    ■ MaleEmployees int   `json:"male_employees"`
    ■ FemaleEmployees int  `json:"female_employees"`
}

// Validate uses a pointer receiver to check the request fields without copying memory.
func (req *CreateBusinessRequest) Validate() error {
    ■ if strings.TrimSpace(req.Name) == "" {
    ■ ■ return errors.New("business name is required")
    ■ }
    ■ if strings.TrimSpace(req.Address) == "" {
    ■ ■ return errors.New("business full address is required")
    ■ }
    ■
    ■ // Check against allowed KP districts
    ■ districtClean := strings.ToLower(strings.TrimSpace(req.District))
    ■ if !allowedDistricts[districtClean] {
    ■ ■ return errors.New("business location district is out of allowed scope")
    ■ }

    ■ if req.MaleEmployees < 0 || req.FemaleEmployees < 0 {
    ■ ■ return errors.New("employee counts cannot be negative")
    ■ }

    ■ return nil
}
```

## Example 2: Handler Executing DTO Validation

This controller handler decodes the request body and executes the validation method:

```
// File: internal/business/handler.go
package business

import (
    ■ "encoding/json"
    ■ "net/http"
    ■ "peace-sme-go/internal/httpx"
)

type Handler struct{}

func (h *Handler) Create(w http.ResponseWriter, r *http.Request) {
    ■ var req CreateBusinessRequest
    ■
    ■ // Decode JSON directly into the struct
    ■ if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
    ■ ■ httpx.WriteError(w, http.StatusBadRequest, "invalid_json", "Failed to parse request JSON")
    ■ ■ return
    ■ }

    ■ // Execute pointer-receiver validation
    ■ if err := req.Validate(); err != nil {
    ■ ■ httpx.WriteError(w, http.StatusUnprocessableEntity, "validation_failed", err.Error())
    ■ ■ return
    ■ }

    ■ // Logic to pass req to Service...
}
}
```

## Service Design

Use a service between handlers and repositories:

```
type Service struct {
    users      UserRepository
    businesses BusinessRepository
    auth       TokenService
}
```

Handlers decode HTTP. Services enforce rules. Repositories run SQL.

## Beginner Walkthrough: POST /api/business

- 1 Vue gathers form fields in reactive state.
- 2 Axios sends JSON with Authorization: Bearer <userToken>.
- 3 Auth middleware verifies JWT and stores user\_id in context.
- 4 Handler decodes JSON into BusinessRequest.
- 5 Service validates district and ownership.
- 6 Repository inserts row into businesses.
- 7 Handler returns {message, business\_id}.
- 8 Vue shows success and redirects to dashboard.

Write this flow in notes before coding. It prevents mixing responsibilities.

## Vue Link

These endpoints power:

- RegistrationForm.vue

- `UserLogin.vue`
- `AppDashboard.vue`
- `SmeBusinessProfile.vue`
- route guards using `userToken`

Keep response shapes stable so the Vue application does not need workaround code.

## Mastery Check

You understand this chapter when you can:

- Explain what receiver parameters are in Go.
- Contrast value receivers and pointer receivers.
- Implement validation routines on request structures using pointer receivers.



# Chapter 8: Grant Applications and Workflow Rules

## Purpose

The grant workflow is the core applicant feature. It teaches larger request payloads, JSONB fields, conditional validation, idempotent updates, and workflow gates. In this chapter, we will study **Interfaces**, understanding how Go implements structural typing (duck typing) without explicit declarations. We will then define **all key application interfaces** used to decouple layers and enable unit testing fakes.

## Foundational Concepts Explained Simply

### 1. Interfaces in Go

#### Deep Dive: Go Interfaces & Implicit Implementation

#### In-Depth Explanation

An **Interface** in Go is a type that declares a contract composed of one or more method signatures.

- **No implements Keyword:** Unlike Java, C#, or TypeScript, you do not explicitly declare that a struct implements an interface.
- **Implicit Satisfaction (Duck Typing):** If a concrete type (like a struct) defines all the methods that an interface requires (with matching names, parameters, and return types), the Go compiler automatically allows that struct to be used wherever that interface is expected.
- **Why This is Powerful:**
  - 1 **Decoupling:** High-level packages can declare interfaces outlining *only* the specific operations they need, rather than importing thick implementation details from lower-level packages.
  - 2 **Unit Testing:** You can easily swap real infrastructure implementations (like an S3 uploader or a PostgreSQL database connection) with lightweight, local testing fakes (mocks) in your test files by simply satisfying the interface methods.

#### Sandbox Program: Mocking an Email Client via Interfaces

This sandbox shows how an HTTP service can call an email notifier interface, allowing us to pass a production SMTP client or a local mock tester for unit testing without changing the service code:

```
package main

import (
    ■ "fmt"
)

// Notifier defines the communication contract
type Notifier interface {
    ■ SendEmail(recipient string, body string) error
}

// ProductionEmailService connects to an external SMTP server
type ProductionEmailService struct {
    ■ APIKey string
}

func (p ProductionEmailService) SendEmail(recipient string, body string) error {
    ■ fmt.Printf("[PROD SMTP] Sending network packet to %s using Brevo Key...\n", recipient)
    ■ return nil
}

// MockEmailService records inputs in-memory for unit testing (no network activity)
type MockEmailService struct {
    ■ SentEmails map[string]string
}
```

```

}

func (m *MockEmailService) SendEmail(recipient string, body string) error {
    m.SentEmails[recipient] = body
    fmt.Printf("[MOCK MAIL] Appended message in memory for %s\n", recipient)
    return nil
}

// NotificationDispatcher depends on the Notifier interface, NOT concrete types
type NotificationDispatcher struct {
    MailClient Notifier
}

func (nd NotificationDispatcher) WelcomeUser(email string) {
    nd.MailClient.SendEmail(email, "Welcome to the PEACE SME Portal!")
}

func main() {
    // 1. Simulate Production Execution
    prodService := ProductionEmailService{APIKey: "sk_brevo_998877"}
    prodDispatcher := NotificationDispatcher{MailClient: prodService}
    prodDispatcher.WelcomeUser("applicant@swat.org")

    fmt.Println()

    // 2. Simulate Unit Test Execution
    mockService := &MockEmailService{SentEmails: make(map[string]string)}
    testDispatcher := NotificationDispatcher{MailClient: mockService}
    testDispatcher.WelcomeUser("test-applicant@shangla.org")

    // Verify the mock state to confirm email was "sent" during assertions
    fmt.Printf("Mock Registry State: Verified sent welcome mail to %s -> %q\n",
        "test-applicant@shangla.org", mockService.SentEmails["test-applicant@shangla.org"])
}

```

## External Resources

- A Tour of Go: Interfaces
- Effective Go: Interfaces and types

## Phased Interfaces Implementation Guide

To master decoupling, we will now define **all core repository and service interfaces** required to run the PEACE SME Grant Portal.

Create a file named `internal/domain/interfaces.go` to declare these interfaces.

### 1. Repository Interfaces

These define operations performed against our PostgreSQL data layer:

```

package domain

import (
    "context"
    "peace-sme-go/internal/db"
)

// UserRepository handles user entity retrieval and updates.
type UserRepository interface {
    FindByID(ctx context.Context, userID int64) (*db.User, error)
    FindByEmail(ctx context.Context, email string) (*db.User, error)
    Create(ctx context.Context, user *db.User) error
    UpdateStatus(ctx context.Context, userID int64, status string) error
}

// BusinessRepository handles business entity operations.
type BusinessRepository interface {
    FindByUserID(ctx context.Context, userID int64) (*db.Business, error)
    Create(ctx context.Context, biz *db.Business) error
    Update(ctx context.Context, biz *db.Business) error
}

```

```
// GrantRepository manages grant workflow operations.
type GrantRepository interface {
    ■FindByUserID(ctx context.Context, userID int64) (*db.Grant, error)
    ■Create(ctx context.Context, grant *db.Grant) error
    ■Update(ctx context.Context, grant *db.Grant) error
    ■UpdateStatus(ctx context.Context, userID int64, status string) error
    ■GetStatus(ctx context.Context, userID int64) (string, error)
}

```

## 2. Infrastructure Service Interfaces

These interfaces decouple our business rules from third-party systems like Redis, S3 storage, and email REST clients:

```
// HFCEnqueuer enqueues asynchronous fraud scoring evaluations.
type HFCEnqueuer interface {
    ■EnqueueRecalculate(ctx context.Context, userID int64) error
}

// MailService handles asynchronous email alerts.
type MailService interface {
    ■SendWelcomeEmail(ctx context.Context, toEmail string, name string) error
    ■SendGrantApprovedEmail(ctx context.Context, toEmail string, approvedAmount float64) error
}

// StorageClient generates upload links and manages asset deletion on S3.
type StorageClient interface {
    ■GeneratePresignedPutURL(ctx context.Context, objectKey string, contentType string) (string, error)
    ■DeleteObject(ctx context.Context, objectURL string) error
}

```

## State Machine Rules

A State Machine enforces valid workflow state changes. In Go, represent this logic as checking current states against target states:

```
// File: internal/grant/status.go
package grant

import "fmt"

type GrantStatus string

const (
    ■StatusDraft      GrantStatus = "Draft"
    ■StatusSubmitted  GrantStatus = "Submitted"
    ■StatusUnderReview GrantStatus = "Under Review"
    ■StatusApproved   GrantStatus = "Approved"
    ■StatusRejected   GrantStatus = "Rejected"
)

func (current GrantStatus) CanTransitionTo(next GrantStatus) error {
    ■switch current {
    ■case StatusDraft:
    ■■if next == StatusSubmitted {
    ■■■return nil
    ■■}
    ■case StatusSubmitted:
    ■■if next == StatusUnderReview || next == StatusApproved || next == StatusRejected {
    ■■■return nil
    ■■}
    ■case StatusUnderReview:
    ■■if next == StatusApproved || next == StatusRejected {
    ■■■return nil
    ■■}
    ■case StatusApproved, StatusRejected:
    ■■return fmt.Errorf("cannot modify status once in terminal state %q", current)
    ■}
    ■return fmt.Errorf("illegal state transition from %q to %q", current, next)
}

```

## Practical Examples

### Example: Decoupled Workflow Execution using Interfaces

This service transitions a grant application to Submitted and enqueues background HFC scoring using injected interface objects:

```
// File: internal/grant/service.go
package grant

import (
    ■"context"
    ■"fmt"
    ■"peace-sme-go/internal/domain"
)

type Service struct {
    ■repo    domain.GrantRepository
    ■hfcSvc  domain.HFCEnqueuer
}

func NewService(repo domain.GrantRepository, hfcSvc domain.HFCEnqueuer) *Service {
    ■return &Service{
    ■■repo:    repo,
    ■■hfcSvc:  hfcSvc,
    ■}
}

func (s *Service) Submit(ctx context.Context, userID int64) error {
    ■statusStr, err := s.repo.GetStatus(ctx, userID)
    ■if err != nil {
    ■■return fmt.Errorf("failed to retrieve status: %w", err)
    ■}

    ■current := GrantStatus(statusStr)
    ■if err := current.CanTransitionTo(StatusSubmitted); err != nil {
    ■■return err
    ■}

    ■if err := s.repo.UpdateStatus(ctx, userID, string(StatusSubmitted)); err != nil {
    ■■return fmt.Errorf("failed to update status: %w", err)
    ■}

    ■// Trigger async scoring via the decoupled interface
    ■if err := s.hfcSvc.EnqueueRecalculate(ctx, userID); err != nil {
    ■■// Log warning but don't fail user request
    ■■fmt.Printf("Warning: failed to enqueue HFC job: %v\n", err)
    ■}

    ■return nil
}
```

## Mastery Check

You understand this chapter when you can:

- Explain Go's implicit satisfaction rule and how it differs from explicit implementation models.
- Declare a clean interface and implement a mock structure that satisfies it.
- Use interface injection to decouple handlers and service layers.
- Build state transition checks enforcing strict workflow rules.

# Chapter 9: Uploads, S3-Compatible Storage, and Media References

## Purpose

The portal stores documents and audio/video media in S3-compatible storage. The database stores references to public object URLs. In this chapter, we will learn about S3 object storage APIs, presigned URLs, and how to safely integrate S3 client packages in Go.

## Foundational Concepts Explained Simply

### 1. Object Storage & Presigned URLs

#### Deep Dive: Object Storage & Presigned URLs

#### In-Depth Explanation

Unlike local file storage which depends on nested folder paths and filesystem drivers, **Object Storage** stores unstructured files flat inside a namespace called a **Bucket**.

- **Object Key:** The unique identifier for a file (e.g. `business_42/cnic_front.pdf`).
- **Metadata:** File descriptors (e.g. `Content-Type: application/pdf`).
- **Presigning URLs:** Standard uploads stream file bytes from the client to the server, which then streams them to S3. This consumes significant server memory and bandwidth.
- To solve this, the server generates a **Presigned URL**.
- The server uses its private S3 API access credentials to compute a cryptographic HMAC signature containing the bucket, key, HTTP method (PUT), and an expiration timestamp.
- The client receives this temporary URL and uploads the file directly to S3 via HTTP PUT.
- The backend never handles the file data stream.

#### Sandbox Program: Presigned URL Mock Signature Generator

This sandbox program demonstrates how a backend computes a signed, expiring URL using HMAC, simulating the exact cryptographic delegation of permissions to a client for direct uploading:

```
package main

import (
    ■ "crypto/hmac"
    ■ "crypto/sha256"
    ■ "encoding/hex"
    ■ "fmt"
    ■ "time"
)

// GenerateMockPresignedURL signs the bucket and key with an expiration window
func GenerateMockPresignedURL(endpoint, bucket, key, method string, expiry time.Time,
    secretKey []byte) string {
    ■ // 1. Format the string that defines the permission scope
    ■ rawURL := fmt.Sprintf("%s/%s/%s", endpoint, bucket, key)
    ■ stringToSign := fmt.Sprintf("METHOD:%s\nURL:%s\nEXPIRY:%d", method, rawURL, expiry.Unix())

    ■ // 2. Generate HMAC signature
    ■ h := hmac.New(sha256.New, secretKey)
    ■ h.Write([]byte(stringToSign))
    ■ signature := hex.EncodeToString(h.Sum(nil))

    ■ // 3. Assemble final URL containing query parameters
```

```

    return fmt.Sprintf("%s?method=%s&expires=%d&signature=%s",
        rawURL, method, expiry.Unix(), signature)
}

func main() {
    endpoint := "https://s3.peace-economic.storage"
    bucket := "grant-uploads"
    key := "business_42/bank_statement.pdf"
    secretKey := []byte("s3-private-storage-secret-key-999")

    // Set URL expiration to 15 minutes in the future
    expiry := time.Now().Add(15 * time.Minute)

    presignedURL := GenerateMockPresignedURL(endpoint, bucket, key, "PUT", expiry, secretKey)
    fmt.Println("Generated Presigned URL for direct file upload:")
    fmt.Println(presignedURL)
}

```

## 2. Upload Validation and Security

### Deep Dive: Upload Validation & Security

#### In-Depth Explanation

- **MIME-Type Verification:** Never rely on the file extension (e.g. .jpg) sent by the client. An attacker could rename a malicious script to exploit.jpg and bypass extension-only filters.
- **Solution:** Read the first 512 bytes of the file and pass it to Go's `http.DetectContentType(...)` function to verify the actual file signature (magic bytes).
- **Path Traversal Prevention:** Always clean custom object names or filesystem targets using `filepath.Base(...)` to strip out path modifiers (like ../) that could let attackers overwrite system files.

#### Sandbox Program: Content Type Detection & Safe Path Base

This program reads mock binary data (magic bytes) to accurately detect MIME types and cleans filenames to prevent path traversal exploits:

```

package main

import (
    "fmt"
    "net/http"
    "path/filepath"
)

func DetectSafeFile(clientFilename string, data []byte) (string, string) {
    // 1. Prevent Path Traversal (e.g., "../../../etc/passwd" -> "passwd")
    safeName := filepath.Base(clientFilename)

    // 2. Detect Content Type from magic bytes (first 512 bytes)
    detectedType := http.DetectContentType(data)

    return safeName, detectedType
}

func main() {
    // A mock malicious upload request
    unsafePath := "../../../var/www/uploads/shell.php.png"

    // PNG Magic bytes: 89 50 4E 47 0D 0A 1A 0A
    pngMagicBytes := []byte{0x89, 0x50, 0x4E, 0x47, 0x0D, 0x0A, 0x1A, 0x0A, 0x00, 0x00}

    safeName, contentType := DetectSafeFile(unsafePath, pngMagicBytes)
    fmt.Println("Original Filename:", unsafePath)
    fmt.Println("Sanitised Safe Filename:", safeName)
    fmt.Println("Detected Content-Type from bytes:", contentType)
}

```

#### External Resources

- [AWS S3 Presigned URL Overview](#)

- MinIO Go Client SDK Documentation
- OWASP File Upload Security Cheat Sheet

## Endpoints

Business:

| Method | Path                                    |
|--------|-----------------------------------------|
| POST   | /api/upload-document/<business_id>      |
| POST   | /api/business/media/generate-upload-url |
| POST   | /api/business/media/save-reference      |

Grant:

| Method | Path                            |
|--------|---------------------------------|
| POST   | /api/grant/generate-upload-url  |
| POST   | /api/grant/save-media-reference |

Applicant status:

| Method | Path                                            |
|--------|-------------------------------------------------|
| POST   | /api/admin/applicant-status/generate-upload-url |

## Two Upload Patterns

The app supports:

- 1 Multipart upload through backend.
- 2 Presigned URL upload directly to S3, followed by saving a DB reference.

Presigned upload is better for large media because the backend does not stream the whole file.

## S3 Concepts

Learn:

- Bucket.
- Object key.
- Content type.
- ACL.
- Presigned PUT URL.
- Public base URL.
- Path-style addressing for S3-compatible providers.

## Database Rows

Business documents:

```
business_documents(document_type, file_name, file_path, mime_type)
```

Grant media:

```
grant_media(media_type, file_name, file_path)
```

`file_path` is a public URL. Your Go storage package should know how to convert an object key into that URL.

## Required Documents

The reports use these required document types:

- CNIC front
- CNIC back
- Business registration certificate
- Tax certificate / NTN
- Bank statement

Make document type values consistent across upload and reporting code.

## Practical Examples

### Example: Generating a Presigned PUT URL in Go

This complete implementation demonstrates how to configure an S3 client and generate a secure presigned URL for direct file uploads:

```
// File: internal/storage/s3.go
package storage

import (
    ■"context"
    ■"fmt"
    ■"time"

    ■"github.com/aws/aws-sdk-go-v2/aws"
    ■"github.com/aws/aws-sdk-go-v2/config"
    ■"github.com/aws/aws-sdk-go-v2/credentials"
    ■"github.com/aws/aws-sdk-go-v2/service/s3"
)

type S3Client struct {
    ■client      *s3.Client
    ■presignClient *s3.PresignClient
    ■bucketName  string
    ■publicBaseURL string
}

type S3Config struct {
    ■EndpointURL  string
    ■AccessKey    string
    ■SecretKey    string
    ■BucketName   string
    ■PublicBaseURL string
}

func NewS3Client(cfg S3Config) (*S3Client, error) {
    ■// Configure credentials and endpoints for S3-compatible providers
    ■customResolver := aws.EndpointResolverWithOptionsFunc(func(service, region string, options ...
    interface{}) (aws.Endpoint, error) {
    ■■return aws.Endpoint{
    ■■■URL:      cfg.EndpointURL,
    ■■■SigningRegion: "us-east-1",
    ■■}, nil
    ■}

    ■sdkCfg, err := config.LoadDefaultConfig(context.Background(),
    ■■config.WithCredentialsProvider(credentials.NewStaticCredentialsProvider(cfg.AccessKey, cfg.
```



```

SecretKey, "")),
■ ■ config.WithEndpointResolverWithOptions(customResolver),
■ )
■ if err != nil {
■ ■ return nil, fmt.Errorf("failed to load S3 SDK config: %w", err)
■ }

■ client := s3.NewFromConfig(sdkCfg, func(o *s3.Options) {
■ ■ o.UsePathStyle = true
■ })
■
■ presignClient := s3.NewPresignClient(client)

■ return &S3Client{
■ ■ client:      client,
■ ■ presignClient: presignClient,
■ ■ bucketName:  cfg.BucketName,
■ ■ publicBaseURL: cfg.PublicBaseURL,
■ }, nil
}

// GeneratePresignedUploadURL creates a temporary PUT URL.
func (s *S3Client) GeneratePresignedUploadURL(ctx context.Context, objectKey string,
contentType string, expiry time.Duration) (string, error) {
■ input := &s3.PutObjectInput{
■ ■ Bucket:      aws.String(s.bucketName),
■ ■ Key:         aws.String(objectKey),
■ ■ ContentType: aws.String(contentType),
■ }

■ presignedReq, err := s.presignClient.PresignPutObject(ctx, input, func(o *s3.
PresignerOptions) {
■ ■ o.Expires = expiry
■ })
■ if err != nil {
■ ■ return "", fmt.Errorf("failed to generate presigned URL: %w", err)
■ }

■ return presignedReq.URL, nil
}

// BuildPublicURL constructs the final public asset address.
func (s *S3Client) BuildPublicURL(objectKey string) string {
■ return fmt.Sprintf("%s/%s", s.publicBaseURL, objectKey)
}

```

## Security Checks

Before saving a reference:

- Verify the authenticated user owns the business.
- Verify the grant belongs to the authenticated user.
- Validate media type as audio or video.
- Validate MIME type allow-lists.
- Avoid trusting client-provided public URLs; prefer object keys and build URLs server-side.

## Mastery Check

You understand this chapter when you can:

- Generate a presigned S3 PUT URL.
- Save an uploaded file reference to PostgreSQL.
- Explain why direct-to-S3 uploads reduce backend load.
- Enforce ownership before saving media.

- Build a public object URL from an object key.

# Chapter 10: Redis, Caching, Access Slots, and Background Jobs

## Purpose

Redis supports three separate concerns in this application: JSON response caching, concurrent applicant access control, and background job queues. In this chapter, we will study **Go Concurrency**, explaining lightweight threads (Goroutines), communication pipes (Channels), and select statements. We will then define **every channel and worker coordination loop** required by the portal.

## Foundational Concepts Explained Simply

### 1. Goroutines

#### Deep Dive: Goroutines: Lightweight Concurrent Threads

#### In-Depth Explanation

In Go, concurrency is achieved using **Goroutines**, which are lightweight, user-space threads managed entirely by the Go runtime scheduler rather than the underlying Operating System.

- **OS Threads vs. Goroutines:** OS threads typically consume about 1MB of memory for their stack space. In contrast, a Goroutine starts with a dynamic, resizable stack of only **2KB**.
- **Spawning:** Prefixing any function call with the `go` keyword schedules the function to run concurrently. The current thread continues executing immediately without blocking.
- **Go Scheduler:** Go uses an M:N scheduler, multiplexing thousands of active Goroutines (N) onto a small number of physical OS threads (M). This eliminates the expensive context-switching overhead of OS thread shifts.

#### Sandbox Program: Spawning Concurrent Tasks

This program demonstrates spawning multiple independent background tasks concurrently using the `go` keyword and coordinates their execution using a simple time delay:

```
package main

import (
    ■ "fmt"
    ■ "time"
)

func runAuditRule(ruleName string, duration time.Duration) {
    ■ fmt.Printf("[HFC AUDIT] Starting risk evaluation: %s\n", ruleName)
    ■ time.Sleep(duration) // Simulate computational rule check
    ■ fmt.Printf("[HFC AUDIT] Finished risk evaluation: %s\n", ruleName)
}

func main() {
    ■ fmt.Println("Main thread: Initiating background evaluation pipeline.")

    ■ // Spawn two background concurrent tasks
    ■ go runAuditRule("Verify CNIC Format", 50*time.Millisecond)
    ■ go runAuditRule("Check Allowed IP Region", 30*time.Millisecond)

    ■ fmt.Println("Main thread: Handlers returned immediately. Sleeping to allow worker tasks to finish...")
    ■
    ■ // Wait long enough for both tasks to complete
    ■ time.Sleep(100 * time.Millisecond)
    ■ fmt.Println("Main thread: Shutting down.")
}
```

## 2. Channels

### Deep Dive: Go Channels & Synchronous Communication

#### In-Depth Explanation

Go's concurrency philosophy is: *"Do not communicate by sharing memory; instead, share memory by communicating."*

- **Channels** (`chan T`): Type-safe pipes that allow separate Goroutines to synchronize and exchange messages without explicit locks or mutexes.
- **Buffered vs. Unbuffered:**
- **Unbuffered** (`make(chan T)`): Sends and receives block until both the sender and receiver are ready. This guarantees synchronous handoffs.
- **Buffered** (`make(chan T, size)`): Sends are non-blocking as long as the buffer is not full. Receives are non-blocking as long as the buffer is not empty.
- **Graceful Channel Closing:** The sender can close a channel using `close(ch)`. Receivers can test if a channel is closed using the comma-ok idiom: `val, ok := <-ch`. If `ok` is `false`, the channel has been drained and closed.

#### Sandbox Program: Channel Coordination and Draining

This program runs a worker that receives work from a buffered channel and communicates results back, showing how channel closes signal completion:

```
package main

import (
    "fmt"
)

func jobProcessor(jobs <-chan int, results chan<- int) {
    for job := range jobs {
        // Process job
        double := job * 2
        results <- double
    }
    close(results) // Close results channel when all jobs are processed
}

func main() {
    jobs := make(chan int, 3)
    results := make(chan int, 3)

    // Start processor
    go jobProcessor(jobs, results)

    // Send 3 jobs
    jobs <- 10
    jobs <- 20
    jobs <- 30
    close(jobs) // Signal that no more jobs will be sent

    // Read results
    for res := range results {
        fmt.Println("Job Result Received:", res)
    }
    fmt.Println("All channel items processed successfully.")
}
```

## 3. Select Statements

### Deep Dive: Select Block Coordination & Timeouts

#### In-Depth Explanation

The `select` statement enables a Goroutine to wait on multiple channel communication operations simultaneously.

- **Multiplexing:** It blocks until one of its cases is ready to send or receive. If multiple cases are ready, it picks one at random.
- **Timeouts:** Combined with `time.After(duration)`, it prevents deadlocks by establishing a maximum wait limit. If no channel responds within the limit, the timeout case fires.
- **Non-blocking Select:** A default case causes the `select` statement to proceed immediately without blocking if no other channels are ready.

## Sandbox Program: Job Queue Coordination with Timeout

This program uses a `select` statement to process messages from multiple queues and includes a timeout protection block:

```
package main

import (
    "fmt"
    "time"
)

func main() {
    emailQueue := make(chan string, 1)
    hfcQueue := make(chan string, 1)

    // Simulate background worker loading
    go func() {
        time.Sleep(20 * time.Millisecond)
        emailQueue <- "Welcome Email for User 42"
    }()

    // Read loop with timeout
    for i := 0; i < 2; i++ {
        select {
        case email := <-emailQueue:
            fmt.Println("Processed Queue Item:", email)
        case hfc := <-hfcQueue:
            fmt.Println("Processed Queue Item:", hfc)
        case <-time.After(50 * time.Millisecond):
            fmt.Println("Job dispatch timed out!")
        }
    }
}
```

## External Resources

- A Tour of Go: Goroutines
- A Tour of Go: Channels
- Go Web Examples: Channels

## Phased Concurrency Implementation Guide

To master concurrency, we will now define **all channel-based background workers and job queues** required by the PEACE SME Grant Portal.

Create a file named `internal/worker/queues.go` to declare these structures.

### 1. Job Structs

First, define the structures representing background tasks:

```
package worker

// EmailJob wraps the parameters needed to send welcome and approval alerts.
type EmailJob struct {
    RecipientEmail string `json:"recipient_email"`
}
```

```

■Subject      string `json:"subject"`
■Body         string `json:"body"`
}

// HFCJob wraps parameters needed to trigger HFC fraud risk scoring.
type HFCJob struct {
■UserID      int64 `json:"user_id"`
■IsTriggeredByAdmin bool `json:"is_triggered_by_admin"`
}

```

## 2. The Worker Manager

The Manager struct acts as a coordinator, maintaining the channels and controlling worker lifecycles:

```

import (
■"context"
■"log"
■"sync"
)

// Manager coordinates job queues and background execution routines.
type Manager struct {
■EmailQueue chan EmailJob // Buffered channel for emails
■HFCQueue   chan HFCJob   // Buffered channel for HFC jobs
■wg         sync.WaitGroup
■shutdown   chan struct{}
}

// NewManager initializes the queues with buffer sizes.
func NewManager(bufferSize int) *Manager {
■return &Manager{
■■EmailQueue: make(chan EmailJob, bufferSize),
■■HFCQueue:   make(chan HFCJob, bufferSize),
■■shutdown:   make(chan struct{}),
■}
}

// Start begins background execution loop goroutines.
func (m *Manager) Start(ctx context.Context) {
■// Start email processor worker
■m.wg.Add(1)
■go m.emailWorker(ctx)

■// Start HFC processor worker
■m.wg.Add(1)
■go m.hfcWorker(ctx)
}

func (m *Manager) emailWorker(ctx context.Context) {
■defer m.wg.Done()
■log.Println("Email worker started.")

■for {
■■select {
■■■case <-ctx.Done():
■■■log.Println("Email worker shutting down...")
■■■return
■■case job, ok := <-m.EmailQueue:
■■■if !ok {
■■■■return
■■■}
■■■m.sendEmail(job)
■■■}
■}
}

func (m *Manager) hfcWorker(ctx context.Context) {
■defer m.wg.Done()
■log.Println("HFC worker started.")

■for {
■■select {
■■■case <-ctx.Done():
■■■log.Println("HFC worker shutting down...")
■■■return
■■case job, ok := <-m.HFCQueue:
■■■if !ok {

```

```

    return
}
m.evaluateHFC(job)
}
}

func (m *Manager) sendEmail(job EmailJob) {
    log.Printf("[EMAIL] Sending message to %s: %s", job.RecipientEmail, job.Subject)
    // actual SMTP/Brevo integration calls...
}

func (m *Manager) evaluateHFC(job HFCJob) {
    log.Printf("[HFC] Re-evaluating rules for User ID %d", job.UserID)
    // actual database/HFC scoring rules checks...
}

// Stop closes channels and blocks until workers drain existing jobs safely.
func (m *Manager) Stop() {
    close(m.EmailQueue)
    close(m.HFCQueue)
    m.wg.Wait()
    log.Println("All background queue workers stopped.")
}

```

## Cache Wrapper

Create a small package `internal/cache/redis.go` to wrap Redis JSON access caching:

```

type Cache struct {
    client *redis.Client
    prefix string
}

```

Methods:

- `GetJSON(ctx, key, &dst)`
- `SetJSON(ctx, key, value, ttl)`
- `DeletePrefix(ctx, prefix)`

Use the configured prefix, defaulting to `peace_sme`.

## Mastery Check

You understand this chapter when you can:

- Explain what goroutines are and why they are lightweight.
- Create unbuffered and buffered channels.
- Coordinate multiple channel reads concurrently using `select`.
- Build a thread-safe worker coordinator using `sync.WaitGroup` to handle graceful shutdowns.

# Chapter 11: Admin, Reports, CSV, and Database Browser

## Purpose

Admin features teach pagination, filtering, sorting, authorization, CSV streaming, and defensive SQL.

## Theoretical Background

### SQL Injection Prevention

SQL Injection occurs when untrusted user input is concatenated directly into SQL query strings, allowing attackers to manipulate queries.

- **Parametric Queries:** Always use parameters (`$1`, `$2` in PostgreSQL) rather than concatenation. The driver sends values to the database engine separately from the SQL command structure, neutralizing threats.
- **Dynamic Sort Columns:** You cannot use query parameters for SQL keywords or schema object names (like columns or tables in `ORDER BY`). To sort dynamically, use a strict **allow-list** mapping user-provided sorting keys to absolute SQL columns.

### Pagination Strategies

- 1 **Offset-Based Pagination (`LIMIT X OFFSET Y`):** Simple to implement and allows direct page jumping.
- **Performance issue:** As `OFFSET` grows, the database must scan and discard all records leading up to the offset, degrading performance on millions of rows.
- 1 **Cursor-Based (Keyset) Pagination (`WHERE id > last_seen_id LIMIT X`):** Highly performant and works on a constant execution plan.
- **Trade-off:** Does not allow jumping directly to page 50 without loading pages 1-49 first.

### Memory-Efficient CSV Streaming

When exporting millions of database rows to a CSV file:

- **Do not** fetch all records into an in-memory array first, as this causes RAM exhaustion.
- Instead, query the database, retrieve a row iterator, and stream rows line-by-line directly to the HTTP response writer. Using `encoding/csv` flushed periodically ensures constant memory usage.

### External Resources

- Use The Index, Luke! - Keyset Pagination Guide
- OWASP SQL Injection Prevention Cheat Sheet
- Go `encoding/csv` package documentation

## Admin Areas

Implement:

- Applicants list.
- Applicant detail dossier.
- Applicant reports.



- Submitted grants report.
- Approved grants report.
- Missing documents report.
- Fully verified report.
- Eligibility criteria report.
- Dashboard stats and frequency charts.
- Updates and FAQ management.
- User blocking and password reset stubs.
- Database browser.

## Pagination Shape

Preserve:

```
{
  "data": [],
  "total": 0,
  "page": 1,
  "per_page": 20
}
```

The frontend depends on this shape.

## Filtering

Reports filter by:

- document status
- language
- gender
- search
- district
- sector
- status
- date range
- HFC status
- amount range

Build filters with parameterized SQL. Never concatenate user input directly into SQL values.

For sort `t_by`, use an allow-list:

```
var allowedSorts = map[string]string{
  "created_at": "u.created_at",
  "district": "b.business_location_district",
}
```

## CSV Exports

CSV export endpoints use a short-lived query token:

- `/api/admin/reports/full-applicant-profiles/csv?token=<query_token>`
- `/api/admin/reports/business-profiles/csv?token=<query_token>`

Do not expose CSV exports with no auth. Query-token auth exists because browsers have trouble attaching bearer headers to direct file downloads.

## Database Browser

Endpoints:

- `GET /api/admin/db-browser/tables`
- `GET /api/admin/db-browser/data/<table_name>`

Security rule:

- Table names must come from `information_schema.tables`.
- Do not let arbitrary path input become raw SQL.

Use an allow-list built from the database table list.

## Content Management

Updates and FAQs are simple CRUD, but they teach cache invalidation:

- Public `/api/updates` and `/api/faqs` can be cached.
- Admin writes must invalidate those cache keys.
- FAQ search can be uncached or cached by query if needed.

## Practical Examples

### Example 1: Defensive Dynamic SQL Builder for Paginated Reports

This example constructs a parameterized query with dynamic filters and validates sorting parameters defensively against SQL injection:

```
// File: internal/report/repository.go
package report

import (
    ■"context"
    ■"fmt"
    ■"strings"

    ■"github.com/jackc/pgx/v5/pgxpool"
)

type ReportFilter struct {
    ■District string
    ■Status   string
    ■Page     int
    ■PerPage  int
    ■SortBy   string
}

// Allow-list for Order By clauses
var allowedSortColumns = map[string]string{
    ■"id":         "g.grant_id",
    ■"amount":     "g.grant_required",
    ■"created_at": "g.application_date",
}
```

```

func (r *Repository) GetGrantsReport(ctx context.Context, f ReportFilter) ([]GrantSummary,
int64, error) {
    ■var conditions []string
    ■var args []interface{}
    ■argCounter := 1

    ■// Add dynamic status filter safely
    ■if f.Status != "" {
        ■conditions = append(conditions, fmt.Sprintf("g.status = %d", argCounter))
        ■args = append(args, f.Status)
        ■argCounter++
    }

    ■// Add dynamic district filter safely
    ■if f.District != "" {
        ■conditions = append(conditions, fmt.Sprintf("g.domicile_district = %d", argCounter))
        ■args = append(args, f.District)
        ■argCounter++
    }

    ■// Build WHERE clause
    ■whereClause := ""
    ■if len(conditions) > 0 {
        ■whereClause = "WHERE " + strings.Join(conditions, " AND ")
    }

    ■// Validate sorting inputs defensively
    ■sortCol, ok := allowedSortColumns[f.SortBy]
    ■if !ok {
        ■sortCol = "g.application_date" // fallback default
    }

    ■// Run total count query
    ■countQuery := fmt.Sprintf("SELECT COUNT(*) FROM grants g %s", whereClause)
    ■var total int64
    ■err := r.db.QueryRow(ctx, countQuery, args...).Scan(&total)
    ■if err != nil {
        ■return nil, 0, fmt.Errorf("failed to count records: %w", err)
    }

    ■// Build main query with Pagination limiters
    ■limit := f.PerPage
    ■offset := (f.Page - 1) * f.PerPage

    ■mainQuery := fmt.Sprintf(`
    ■SELECT g.grant_id, g.user_id, g.status, g.grant_required
    ■FROM grants g
    ■%s
    ■ORDER BY %s DESC
    ■LIMIT %d OFFSET %d`, whereClause, sortCol, argCounter, argCounter+1)

    ■args = append(args, limit, offset)

    ■// Execute mainQuery and scan rows...
    ■return summaries, total, nil
}

```

## Example 2: Streaming CSV Direct to Client

This example streams database records as CSV directly into an HTTP response using constant memory overhead:

```

// File: internal/report/handler.go
package report

import (
    ■"encoding/csv"
    ■"log"
    ■"net/http"
    ■"strconv"
)

type Handler struct {
    ■db *pgxpool.Pool
}

func (h *Handler) StreamCSVExport(w http.ResponseWriter, r *http.Request) {
    ■// Set correct response headers for direct download

```

```

■w.Header().Set("Content-Type", "text/csv")
■w.Header().Set("Content-Disposition", "attachment;filename=grants_export.csv")

■// Instantiate standard Go CSV writer writing directly to the ResponseWriter
■writer := csv.NewWriter(w)
■defer writer.Flush()

■// Write CSV headers
■if err := writer.Write([]string{"Grant ID", "User ID", "Amount Requested"}); err != nil {
■■log.Printf("Failed to write CSV header: %v", err)
■■return
■}

■// Open connection and iterate rows row-by-row
■rows, err := h.db.Query(r.Context(), "SELECT grant_id, user_id, grant_required FROM grants")
■if err != nil {
■■http.Error(w, "database read failure", http.StatusInternalServerError)
■■return
■}
■defer rows.Close()

■for rows.Next() {
■■var grantID, userID int64
■■var amount float64
■■if err := rows.Scan(&grantID, &userID, &amount); err != nil {
■■■log.Printf("Failed to scan row: %v", err)
■■■return
■■}

■■record := []string{
■■■strconv.FormatInt(grantID, 10),
■■■strconv.FormatInt(userID, 10),
■■■strconv.FormatFloat(amount, 'f', 2, 64),
■■}

■■if err := writer.Write(record); err != nil {
■■■log.Printf("Failed to write CSV row: %v", err)
■■■return
■■}
■■
■■// Flush buffer to client periodically
■■writer.Flush()
■}
}

```

## Mastery Check

You understand this chapter when you can:

- Build paginated SQL with total count.
- Add filters without SQL injection.
- Stream a CSV response.
- Protect admin-only routes.
- Invalidate cache after content writes.
- Build a database browser without allowing arbitrary SQL execution.

# Chapter 12: HFC Fraud Detection and Approval Authority

## Purpose

HFC teaches deterministic scoring, background recalculation, audit trails, and role-based approvals.

## Theoretical Background

### Rule-Based Engines vs. ML Models

In fraud detection and risk scoring, systems use one of two main approaches:

- 1 **Deterministic Rule-Based Engines:** Apply strict, logical heuristics to evaluate input data (e.g., "If email already exists, add 20 points").
  - **Pros:** 100% transparent, predictable, easy to debug, and requires no training datasets.
  - **Cons:** Rigid; cannot easily detect novel fraud patterns that do not violate explicit rules.
- 1 **Machine Learning Classifiers:** Make probabilistic predictions based on feature sets learned from historical data.
  - **Pros:** Can detect complex, non-linear fraud signals.
  - **Cons:** Hard to explain (black-box problem), prone to data drift, and slower to execute.

This application utilizes a deterministic rule engine (v1) as its primary scorer, with optional hook points for ML scoring.

### Message Processing Guarantees

When background workers process jobs (such as recalculating HFC scores or sending approval notifications), three execution guarantees are possible:

- **At-Most-Once:** The job is pulled from the queue and immediately marked complete before execution. If the worker crashes mid-job, the task is lost.
- **At-Least-Once (Recommended):** The job remains in the queue (or in a processing state) until the worker explicitly acknowledges completion. If the worker crashes, the job is re-run. This requires writing **Idempotent** tasks (tasks that can run multiple times safely without duplicate side-effects).
- **Exactly-Once:** Achieved by combining At-Least-Once processing with deduplication mechanisms (like database unique indexes or transaction locks).

### Transactional Outbox Pattern

When performing actions that involve writing to a database and triggering an external side effect (like sending an email or publishing to a message broker):

- **Do not** call the external service inside the database transaction. If the service is slow, it holds the DB connection open. If the transaction rolls back, the email cannot be unsent (dual-write problem).
- **Solution:** Use the Transactional Outbox pattern. Write both the primary state change (e.g., ApproveGrant) and a task record (e.g., SendEmailJob) to the database inside the same transaction. A separate background worker reads the outbox table and executes the side-effects.

### External Resources

- Distributed Systems - Messaging Guarantees

- Microservices Pattern: Transactional Outbox
- Wikipedia: Rule-Based System

## HFC Rules

Score rules:

| Rule                            | Points |
|---------------------------------|--------|
| Duplicate CNIC                  | 50     |
| Duplicate email                 | 20     |
| Duplicate mobile                | 20     |
| Missing business profile        | 30     |
| Missing required documents      | 25     |
| Missing grant media             | 10     |
| Business district out of scope  | 40     |
| Grant amount above threshold    | 15     |
| Application submitted very fast | 10     |
| Missing expression of interest  | 15     |

Risk bands:

| Score | Level    |
|-------|----------|
| 0-29  | LOW      |
| 30-59 | MEDIUM   |
| 60-79 | HIGH     |
| 80+   | CRITICAL |

## Tables

HFC writes:

- hfc\_evaluations
- hfc\_review\_actions
- HFC fields on grants

Approval writes:

- grants.status
- grants.approved\_amount
- grants.approval\_reason
- grants.approved\_at
- grants.approved\_by
- grant\_approval\_logs

## Admin Endpoints

HFC:

- GET /api/admin/hfc/dashboard/stats
- GET /api/admin/hfc/queue
- GET /api/admin/hfc/applicant/<user\_id>
- POST /api/admin/hfc/applicant/<user\_id>/action
- POST /api/admin/hfc/applicant/<user\_id>/recalculate

Approval:

- GET /api/admin/grants/<user\_id>/approval-check
- POST /api/admin/grants/<user\_id>/approve

## Shadow Mode

If HFC\_SHADOW\_MODE=1, HFC score does not block approval. The score is advisory.

If shadow mode is off, define explicit blocking behavior before implementation. For example, require override for HIGH/CRITICAL risk.

## Audit Trails

Every admin action should record:

- actor username
- action type
- before state
- after state
- comment
- timestamp

Audit records are not decoration. They are how an approval system explains itself later.

## Practical Examples

### Example: Implementing a Deterministic HFC Scorer in Go

This example shows how to write a scoring engine that queries various database tables and calculates a cumulative fraud score based on the application rules:

```
// File: internal/hfc/scorer.go
package hfc

import (
    ■ "context"
    ■ "fmt"
)

type EvaluationResult struct {
    ■ Score      int
    ■ RiskLevel  string
    ■ Details    map[string]int
}

type ScorerRepository interface {
    ■ CheckDuplicateCNIC(ctx context.Context, userID int64, cnic string) (bool, error)
```

```

■HasBusinessProfile(ctx context.Context, userID int64) (bool, error)
■GetUploadedDocumentsCount(ctx context.Context, userID int64) (int, error)
}

type Scorer struct {
■repo ScorerRepository
}

func NewScorer(repo ScorerRepository) *Scorer {
■return &Scorer{repo: repo}
}

// Evaluate calculates the risk score and risk bands for an applicant.
func (s *Scorer) Evaluate(ctx context.Context, userID int64, cnic string) (*EvaluationResult,
error) {
■score := 0
■details := make(map[string]int)

■// Rule 1: Check for duplicate CNIC
■isDuplicateCNIC, err := s.repo.CheckDuplicateCNIC(ctx, userID, cnic)
■if err != nil {
■■return nil, fmt.Errorf("failed to check duplicate CNIC: %w", err)
■}
■if isDuplicateCNIC {
■■score += 50
■■details["duplicate_cnic"] = 50
■}

■// Rule 2: Check for missing business profile
■hasProfile, err := s.repo.HasBusinessProfile(ctx, userID)
■if err != nil {
■■return nil, fmt.Errorf("failed to check business profile: %w", err)
■}
■if !hasProfile {
■■score += 30
■■details["missing_business_profile"] = 30
■}

■// Rule 3: Check for incomplete documents (expecting 5 required uploads)
■uploadedCount, err := s.repo.GetUploadedDocumentsCount(ctx, userID)
■if err != nil {
■■return nil, fmt.Errorf("failed to get docs count: %w", err)
■}
■if uploadedCount < 5 {
■■score += 25
■■details["missing_required_documents"] = 25
■}

■// Determine Risk Level Band
■var riskLevel string
■switch {
■case score <= 29:
■■riskLevel = "LOW"
■case score <= 59:
■■riskLevel = "MEDIUM"
■case score <= 79:
■■riskLevel = "HIGH"
■default:
■■riskLevel = "CRITICAL"
■}

■return &EvaluationResult{
■■Score:    score,
■■RiskLevel: riskLevel,
■■Details:  details,
■}, nil
}

```

## Mastery Check

You understand this chapter when you can:

- Implement deterministic scoring from database facts.



- Store a full HFC evaluation record.
- Update grant HFC fields from latest evaluation.
- Restrict approval to `is_approver`.
- Record approval logs.
- Explain why shadow mode changes enforcement but not scoring.

# Chapter 13: Vue 3 Frontend Foundations

## Purpose

The Vue frontend teaches SPA structure, routing, state stored in localStorage, Axios API calls, bilingual rendering, and component composition.

Application parallel: every Vue concept in this chapter is tied to a PEACE SME screen. You learn `ref` by storing login fields, `reactive` by building the business profile form, `computed` by switching English/Urdu labels, and `route guards` by protecting applicant/admin pages.

## Foundational Concepts Explained Simply

### 1. SPA vs. MPA Architecture

#### Deep Dive: SPA vs. MPA Architecture

#### In-Depth Explanation

- **Multi-Page Applications (MPAs):** Traditional sites (e.g., standard Flask or Laravel setups) request a new HTML page from the server on every navigation. The browser window flashes white, stylesheet assets are parsed again, and any local client-side memory state is wiped out.
- **Single Page Applications (SPAs):** Load a single `index.html` shell containing a JavaScript bundle.
- **Routing:** Navigation is intercepted client-side via the HTML5 History API (`history.pushState`). The URL path changes, but the browser does not reload. Instead, a routing library (like Vue Router) swaps the active component layout in the DOM.
- **Data Flow:** The frontend communicates with the Go backend asynchronously by fetching JSON payloads via HTTP requests (Axios or Fetch).

#### Sandbox Program: Simple Vanilla Client-Side Router

This sandbox demonstrates how client-side routing works in vanilla JavaScript. It intercepts click events, updates the URL history, and dynamically swaps out page views:

```
<!-- Vanilla HTML SPA Simulator -->
<div id="app">
  <nav>
    <a href="/" onclick="navigate(event, '/')">Home</a> |
    <a href="/about" onclick="navigate(event, '/about')">About</a>
  </nav>
  <div id="view" style="padding: 20px; border: 1px solid #ccc; margin-top: 10px;"></div>
</div>

<script>
// Mock view layouts mapping paths to templates
const routes = {
  '/': '<h1>Home Page</h1><p>Welcome to PEACE SME SPA!</p>',
  '/about': '<h1>About Us</h1><p>We empower regional business rewrite systems.</p>'
};

function renderView(path) {
  const viewElement = document.getElementById('view');
  viewElement.innerHTML = routes[path] || '<h1>404 Not Found</h1>';
}

function navigate(event, path) {
  event.preventDefault(); // Prevent full page browser reload
  window.history.pushState({}, '', path); // Update address bar
  renderView(path); // Update DOM dynamically
}

// Handle browser Back/Forward navigation triggers
window.onpopstate = () => {
```

```

    renderView(window.location.pathname);
  };

  // Initial Render
  renderView('/');
</script>

```

## 2. Vue 3 Reactivity System (ES6 Proxies)

### Deep Dive: Vue 3 Reactivity & ES6 Proxies

#### In-Depth Explanation

Vue 3's reactivity system tracks variable changes and automatically updates the HTML DOM.

- **Reactivity Mechanism:** Built using native **ES6 Proxies**. When you declare a reactive target via `ref(value)` or `reactive(object)`, Vue wraps the target object in a Proxy instance.
- **Get Trap (Dependency Tracking):** When a component template renders and accesses a reactive property, the Proxy's get handler is triggered. Vue registers the calling component as a dependency of that property.
- **Set Trap (DOM Re-rendering):** When the property is modified, the Proxy's set handler intercepts the write. It notifies all registered dependency functions to execute, causing the Virtual DOM to compare changes and re-render.

#### Sandbox Program: Custom Reactivity in Vanilla JavaScript

This sandbox implements a lightweight version of Vue's `ref()` function using ES6 Proxies to show how visual DOM updates are triggered whenever a JavaScript property changes:

```

<!-- Simple Vanilla Reactivity Simulator -->
<div class="card p-4">
  <h2 id="counter-title">Count: 0</h2>
  <button onclick="increment()">Add 1</button>
</div>

<script>
// Mock Vue "ref" implementation using ES6 Proxies
function myRef(initialValue) {
  const container = { value: initialValue };

  // Trackers list (effects to run when value changes)
  const subscribers = [];

  return new Proxy(container, {
    get(target, key) {
      if (key === 'value' && activeEffect) {
        subscribers.push(activeEffect); // Track dependency
      }
      return target[key];
    },
    set(target, key, newValue) {
      if (key === 'value') {
        target[key] = newValue;
        // Trigger all dependent effects
        subscribers.forEach(effect => effect());
      }
      return true;
    }
  });
}

// Global active effect tracker
let activeEffect = null;

// The reactive state variable
const count = myRef(0);

// Register a renderer effect (acts like Vue's component compile render)
activeEffect = () => {
  document.getElementById('counter-title').innerText = `Count: ${count.value}`;
};

```

```

activeEffect(); // Run once initially to register get hook
activeEffect = null; // Clear active tracker

function increment() {
  count.value++; // Mutating value automatically triggers set and updates DOM!
}
</script>

```

### 3. Composition API vs. Options API

#### Deep Dive: Composition API vs. Options API

##### In-Depth Explanation

- **Options API:** Features are organized by option blocks: `data()`, `methods`, `computed`, `watch`. In complex components with multiple features, code for a single feature is scattered across multiple locations, making it difficult to maintain.
- **Composition API** (`<script setup>`): Organizes code by logical feature concern in a unified script block.
- **Composables:** Allows you to extract reactive state and business logic into separate, reusable files (e.g. `useAuth.js`) and share them across different components.

##### Sandbox Comparison: Options API vs. Composition API

Compare these two component styles performing the exact same logic. Note how clean, organized, and reusable the Composition API layout is:

##### Options API Style

```

export default {
  data() {
    return {
      language: 'english'
    }
  },
  computed: {
    isUrdu() {
      return this.language === 'urdu'
    }
  },
  methods: {
    toggleLang() {
      this.language = this.language === 'english' ? 'urdu' : 'english'
    }
  }
}

```

##### Composition API Style (Modular and reusable)

```

<script setup>
import { ref, computed } from 'vue'

// Local state
const language = ref('english')

// Computed properties
const isUrdu = computed(() => language.value === 'urdu')

// Actions
function toggleLang() {
  language.value = language.value === 'english' ? 'urdu' : 'english'
}
</script>

```

##### External Resources

- [Vue.js Official Guide: Reactivity in Depth](#)
- [Vue Router Documentation](#)
- [Axios Getting Started Guide](#)

## Core Vue Concepts

Learn:

- createApp
- Single File Components
- Composition API
- ref
- reactive
- computed
- watch
- onMounted
- props and emits
- Vue Router
- Axios interceptors
- Tailwind utility classes

## Concept to Portal Feature

| Vue concept       | PEACE SME use                                |
|-------------------|----------------------------------------------|
| ref               | login email, password, loading, error        |
| reactive          | business profile form and grant form         |
| computed          | selected translation object and RTL state    |
| watch             | reload admin reports when filters change     |
| onMounted         | fetch dashboard profile and announcements    |
| props             | reusable input, upload, and table components |
| emits             | upload complete, modal close, row selected   |
| router guard      | redirect unauthenticated users               |
| Axios interceptor | attach JWT bearer token                      |

## Application Entry

Recommended structure:

```
frontend/src/  
  main.js  
  App.vue  
  apiConfig.js  
  router/index.js  
  api/client.js  
  views/  
  components/
```

apiConfig.js is the single source of truth for the backend base URL.

## Route Guards

Preserve:

```
requiresAuth -> localStorage.userToken -> else /login
requiresAdmin -> localStorage.adminToken -> else /admin/login
```

Do not rename token keys unless you update all dependent code.

Application parallel:

- /dashboard requires userToken.
- /business-profile requires userToken.
- /grant-application requires userToken.
- /admin/dashboard requires adminToken.
- /admin/grants/submitted/:id requires adminToken.

Frontend guards improve navigation, but the Go backend remains the real security boundary.

## Axios Client

Create one Axios instance. Attach tokens based on route area or helper calls:

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('userToken')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

For admin calls, either use a second admin client or explicit helper that attaches adminToken.

## Bilingual UI

Language storage:

```
localStorage.setItem('language', 'urdu')
```

Computed state:

```
const isUrdu = computed(() => localStorage.getItem('language') === 'urdu')
```

Rendering:

```
<h1 :class="{ 'text-right font-urdu': isUrdu }">{{ t.title }}</h1>
```

Store values in English even when labels render in Urdu.

## Tailwind Discipline

Use Tailwind for layout, spacing, typography, and responsive states. Keep repeated visual patterns in components:

- Navbar.vue
- AdminNavbar.vue
- Footer.vue
- drawers
- modals
- upload controls
- table controls

## Practical Examples

### Example 1: Vue 3 Single File Component (SFC)

This complete example demonstrates the Composition API (<script setup>) syntax, reactivity (ref, computed), lifecycle hooks (onMounted), and dynamic RTL layouts for Urdu translation:

```
<!-- File: src/views/WelcomeBanner.vue -->
<script setup>
import { ref, computed, onMounted } from 'vue'

// Reactive state
const name = ref('')
const language = ref(localStorage.getItem('language') || 'english')

// Translation object
const translations = {
  english: {
    welcome: 'Welcome to PEACE SME Grant Portal',
    placeholder: 'Enter your name',
  },
  urdu: {
    welcome: '■■ ■■ ■■ ■■ ■■ ■■ ■■ ■■■■■ ■■■■■ ■■ ■■ ■■■■■',
    placeholder: '■■■■ ■■■ ■■■ ■■■■',
  }
}

// Computed property for language detection
const isUrdu = computed(() => language.value === 'urdu')

// Computed translations
const t = computed(() => translations[language.value])

// Lifecycle Hook
onMounted(() => {
  console.log('Welcome component mounted successfully.')
})
</script>

<template>
  <div :class="[isUrdu ? 'text-right rtl font-urdu' : 'text-left ltr', 'p-6 bg-white shadow rounded-lg']">
    <h1 class="text-2xl font-bold text-slate-800 mb-4">
      {{ t.welcome }} <span v-if="name" class="text-blue-600">{{ name }}</span>
    </h1>

    <label class="block text-sm font-medium text-gray-700 mb-2">{{ t.placeholder }}</label>
    <input
      v-model="name"
      type="text"
      class="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md focus:outline-none focus:ring-blue-500 focus:border-blue-500"
      :placeholder="t.placeholder"
    />
  </div>
</template>

<style scoped>
.font-urdu {
  font-family: 'Noto Nastaliq Urdu', sans-serif;
  line-height: 2.2;
}
</style>
```

### Example 2: Configuring Axios Interceptors globally

This example shows how to set up an Axios instance that intercepts all requests to inject bearer tokens from localStorage dynamically:

```
// File: src/api/client.js
import axios from 'axios'
import apiConfig from '../apiConfig'

const apiClient = axios.create({
  baseURL: apiConfig.baseURL,
```

```

    timeout: 10000, // 10 seconds
    headers: {
      'Content-Type': 'application/json'
    }
  })

// Request Interceptor: Attach authentication tokens dynamically
apiClient.interceptors.request.use(
  (config) => {
    // Check if route is admin, use admin token; else use user token
    const isAdminRoute = window.location.pathname.startsWith('/admin')
    const tokenKey = isAdminRoute ? 'adminToken' : 'userToken'
    const token = localStorage.getItem(tokenKey)

    if (token) {
      config.headers.Authorization = `Bearer ${token}`
    }

    return config
  },
  (error) => {
    return Promise.reject(error)
  }
)

// Response Interceptor: Handle errors (like 401 Unauthorized) globally
apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      log.error('Unauthorized access detected. Redirecting to login.')
      // Invalidate tokens and redirect
      localStorage.removeItem('userToken')
      localStorage.removeItem('adminToken')
      window.location.href = '/login'
    }
    return Promise.reject(error)
  }
)

export default apiClient

```

## Mastery Check

You understand this chapter when you can:

- Build a route-protected Vue page.
- Fetch backend JSON with Axios.
- Store and read JWTs from localStorage.
- Switch English/Urdu labels without changing stored values.
- Use Composition API instead of Options API for new code.



# Chapter 14: Vue Applicant and Admin Interfaces

## Purpose

This chapter turns Vue fundamentals into full application screens.

## Theoretical Background

### Form State Management (Wizard Pattern)

Large multi-step forms (like the 9-section Grant Application form) require strict state management:

- **Single Source of Truth:** Keep form data in a single, root-level reactive object (`reactive()`). This avoids syncing child component state manually.
- **Wizard Flow Patterns:** Break the layout into steps. A reactive `currentStep` index determines which section renders (`v-if="currentStep === X"`).
- **Validation Gates:** Before allowing a user to progress to the next step, execute validation logic on the subfields of the current step.
- **Dirty Checking:** Track if the user has modified fields without saving, allowing you to prompt a warning before they close the tab or navigate away.

### Chart Rendering and Component Cleanup

When rendering visualizations (like frequency reports or risk distributions) using Canvas-based libraries (Chart.js):

- Canvas rendering requires a physical DOM node to exist on the page. Therefore, chart initialization must take place inside the `onMounted()` hook.
- **Preventing Memory Leaks:** If a user navigates to another page, the component is unmounted but the chart instance may still exist in heap memory. Always store a reference to the chart instance and destroy it in the `onBeforeUnmount()` lifecycle hook.

### External Resources

- [Vue.js State Management Guide](#)
- [Chart.js Integration with Vue](#)

## Public Views

Build:

- Landing page with updates and FAQ access.
- Language selection.
- Terms and conditions.
- Registration form.
- User login.
- Admin login.
- Waiting room.
- Geo-blocked page.
- Password reset disabled pages.

## Applicant Views

Build:

- Dashboard.
- Business profile form.
- Grant application form.
- Grant review page.
- Upload flows for documents and media.

The grant form is large. Split it into sections:

- 1 Applicant identification.
- 2 Business details.
- 3 Purpose of grant.
- 4 Items to be financed.
- 5 Contribution.
- 6 Business growth.
- 7 Disclaimer.
- 8 Declaration.
- 9 How did you hear.

Use child components when a section has independent state and validation.

## Admin Views

Build:

- Admin dashboard.
- Submitted grants table.
- Grant detail.
- Applicant status.
- Grant access whitelist.
- Applicant report.
- Applied details report.
- Approved grants report.
- Missing documents report.
- Eligibility criteria report.
- Full report.
- HFC dashboard, queue, applicant detail, model tuning.
- Updates management.
- FAQ management.
- Database browser.

## Form State Pattern

Use:

```
const form = reactive({
  grant_required: null,
  financed_items: [],
  declaration_accepted: false
})
```

For repeating rows:

```
function addItem() {
  form.financed_items.push({ item: '', quantity: 1, estimated_cost: null })
}
```

## Admin Tables

Tables need:

- loading state
- error state
- empty state
- pagination controls
- sortable headers
- filters
- CSV export action where supported

Keep query state in the URL when useful so admins can refresh or share filtered views.

## Charts

Use Chart.js for:

- dashboard counts
- registration/application frequency
- HFC status distribution
- eligibility criteria breakdown

Charts should be backed by API responses, not hardcoded data.

## Practical Examples

### Example: Multi-step Wizard Form with Validation and State

This example demonstrates a complete Vue component layout implementing a multi-step form wizard with inline step validation and repeating row creation:

```
<!-- File: src/components/GrantWizard.vue -->
<script setup>
import { ref, reactive, computed } from 'vue'

const currentStep = ref(1)

// Root Single Source of Truth form state
const form = reactive({
```

```

    applicantName: '',
    financedItems: [
      { name: '', qty: 1, cost: 0 }
    ]
  })

// Validation per step
const stepErrors = ref([])

const isStepValid = () => {
  stepErrors.value = []

  if (currentStep.value === 1) {
    if (!form.applicantName.trim()) {
      stepErrors.value.push('Applicant name is required.')
    }
  }

  if (currentStep.value === 2) {
    if (form.financedItems.length === 0) {
      stepErrors.value.push('At least one item must be added.')
    }
    for (let i = 0; i < form.financedItems.length; i++) {
      if (!form.financedItems[i].name.trim() || form.financedItems[i].cost <= 0) {
        stepErrors.value.push(`Item ${i + 1} has invalid name or estimated cost.`)
      }
    }
  }

  return stepErrors.value.length === 0
}

const nextStep = () => {
  if (isStepValid()) {
    currentStep.value++
  }
}

const prevStep = () => {
  currentStep.value--
}

// Add/Remove repeating item rows
const addRow = () => {
  form.financedItems.push({ name: '', qty: 1, cost: 0 })
}

const removeRow = (index) => {
  form.financedItems.splice(index, 1)
}

// Compute total requested cost
const totalCost = computed(() => {
  return form.financedItems.reduce((sum, item) => sum + (item.qty * item.cost), 0)
})

const submitForm = () => {
  if (isStepValid()) {
    console.log('Submitting payload to /api/grant:', form)
    // api.post('/api/grant', form) ...
  }
}
</script>

<template>
  <div class="max-w-xl mx-auto p-8 bg-white border border-slate-200 rounded-xl shadow-lg">
    <!-- Step Indicators -->
    <div class="flex items-center justify-between mb-8">
      <span :class="['font-bold', currentStep >= 1 ? 'text-blue-600' : 'text-slate-400']">1. Identity</span>
      <span class="w-12 h-0.5 bg-slate-300"></span>
      <span :class="['font-bold', currentStep >= 2 ? 'text-blue-600' : 'text-slate-400']">2. Financed Items</span>
    </div>

    <!-- Validation Warnings -->
    <div v-if="stepErrors.length > 0" class="mb-6 p-4 bg-red-50 border-l-4 border-red-500 text-

```

```

red-700">
  <ul>
    <li v-for="err in stepErrors" :key="err" class="text-sm">{{ err }}</li>
  </ul>
</div>

<!-- Step 1 View -->
<div v-if="currentStep === 1" class="space-y-4">
  <label class="block text-sm font-semibold text-slate-700">Full Name of Applicant</label>
  <input
    v-model="form.applicantName"
    type="text"
    class="w-full px-4 py-2 border border-slate-300 rounded-lg focus:ring-2 focus:ring-
blue-500"
  />
</div>

<!-- Step 2 View -->
<div v-if="currentStep === 2" class="space-y-4">
  <h3 class="text-lg font-bold text-slate-800">Items to be Financed</h3>

  <div v-for="(item, idx) in form.financedItems" :key="idx" class="flex gap-4 items-
center">
    <input
      v-model="item.name"
      placeholder="Item name"
      class="flex-1 px-3 py-2 border rounded-md"
    />
    <input
      v-model.number="item.qty"
      type="number"
      placeholder="Qty"
      class="w-16 px-3 py-2 border rounded-md"
    />
    <input
      v-model.number="item.cost"
      type="number"
      placeholder="Cost"
      class="w-24 px-3 py-2 border rounded-md"
    />
    <button
      @click="removeRow(idx)"
      class="text-red-500 font-bold hover:text-red-700"
    >
      &times;
    </button>
  </div>

  <button
    @click="addRow"
    class="text-sm text-blue-600 hover:underline"
  >
    + Add Another Item
  </button>

  <div class="pt-4 border-t text-right font-bold text-slate-800">
    Total Estimated Cost: ${{ totalCost }}
  </div>
</div>

<!-- Navigation Buttons -->
<div class="mt-8 flex justify-between">
  <button
    v-if="currentStep > 1"
    @click="prevStep"
    class="px-4 py-2 bg-slate-100 hover:bg-slate-200 rounded-lg"
  >
    Back
  </button>
  <div class="ml-auto">
    <button
      v-if="currentStep < 2"
      @click="nextStep"
      class="px-5 py-2 bg-blue-600 hover:bg-blue-700 text-white rounded-lg"
    >
      Next
    </button>
  </div>
</div>

```

```
<button
  v-else
  @click="submitForm"
  class="px-5 py-2 bg-emerald-600 hover:bg-emerald-700 text-white rounded-lg"
>
  Submit Application
</button>
</div>
</div>
</div>
</template>
```

## Mastery Check

You understand this chapter when you can:

- Build a large form without losing state.
- Create reusable admin table controls.
- Keep Vue labels bilingual.
- Connect every screen to the matching `/api` endpoint.
- Handle loading, error, empty, and success states professionally.

# Chapter 15: Testing, Debugging, and API Compatibility

## Purpose

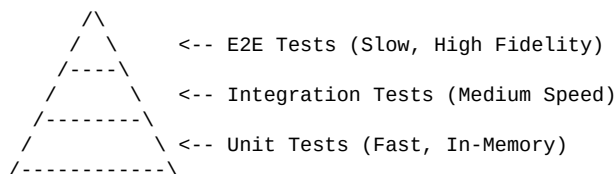
Testing proves that the Go and Vue rewrite matches the Flask behavior. Compatibility is the product.

## Theoretical Background

### The Testing Pyramid

A balanced software testing strategy is represented as a pyramid:

- 1 **Unit Tests (Base):** Focus on isolated functions (e.g., config loaders, score calculators). They are fast, reliable, and run entirely in memory.
- 2 **Integration Tests (Middle):** Validate that multiple components (e.g., service layers interacting with a real PostgreSQL connection pool or S3 mock) work together correctly.
- 3 **End-to-End (E2E) Tests (Top):** Spin up the entire system (Vue frontend + Go backend + DB + Redis) and simulate real user flows via browser automated drivers (like Cypress or Playwright).



### Table-Driven Testing in Go

Go's idiomatic pattern for unit testing is **Table-Driven Testing**:

- You define a slice of anonymous structs representing input parameters, expected outputs, and test description names.
- You iterate over this "table" of test cases, running each case as an isolated subtest using `t.Run()`.
- This keeps test files clean, makes it easy to add new test vectors, and avoids repetitive copy-paste testing code.

### Keyset Regression & API Compatibility

When migrating a backend:

- The HTTP path, request verbs, response JSON keys, and HTTP status codes must match the original implementation exactly.
- Test suites must verify contract details: e.g., verifying that a missing field returns exactly `422 Unprocessable Entity` containing the specific error keys expected by Axios interceptors.

### Git Bisect for Debugging Regressions

When a regression bug is introduced in a codebase and it is unclear which commit caused it:

- **Git Bisect** executes a binary search through your commit history.
- You mark a known "good" commit (where the bug did not exist) and a known "bad" commit (where the bug exists).
- Git checks out intermediate commits, prompting you (or an automated test script) to classify each commit as good or bad, rapidly locating the root cause commit.

### External Resources

- Go Wiki: Table-Driven Tests
- Git Documentation: git-bisect
- Cypress End-to-End Testing Guide

## Go Test Layers

Use:

- Unit tests for config, validation, JWT, HFC scoring.
- Repository tests for SQL queries.
- Handler tests for HTTP status and JSON shape.
- Integration tests for full workflows.

## Handler Tests

Use `httptest`:

```
req := httptest.NewRequest(http.MethodGet, "/api/updates", nil)
rr := httptest.NewRecorder()
handler.ServeHTTP(rr, req)
```

Assert:

- status code
- content type
- JSON fields
- error shape

## Compatibility Tests

For each endpoint, test:

- method
- path
- auth requirement
- request payload
- response shape
- important business rules

Examples:

- Registration closed returns 403.
- Blocked user cannot login.
- Grant submission requires whitelist when enabled.
- Non-approver cannot approve grant.
- HFC shadow mode does not block approval.

## Vue Testing



Use component tests for:

- route guard behavior
- login form submission
- language switching
- grant form conditional sections
- admin table filters

Use end-to-end tests for:

- login to dashboard
- create/update business profile
- admin login and report loading
- grant application submission path

## Debugging Tools

Backend:

```
go test ./...
go test ./internal/grant -run TestGrantAccess -v
go test -race ./...
```

Frontend:

```
npm run test
npm run build
```

Git:

```
git diff
git blame path/to/file
git bisect start
```

## Practical Examples

### Example 1: Table-Driven Unit Test in Go

This complete testing code demonstrates table-driven unit testing using Go subtests for a business registration validation helper:

```
// File: internal/business/dto_test.go
package business

import (
    "testing"
)

func TestCreateBusinessRequest_Validate(t *testing.T) {
    // Table of test cases
    tests := []struct {
        name      string
        request    CreateBusinessRequest
        expectError bool
        errorMsg   string
    }{
        {
            name: "Valid request",
            request: CreateBusinessRequest{
                Name: "Khyber Tech Solutions",
                Address: "Main Bazaar, Swat",
                District: "Swat",
                MaleEmployees: 5,
            },
            expectError: false,
            errorMsg: "",
        },
    }

    for _, test := range tests {
        t.Run(test.name, func(t *testing.T) {
            err := test.request.Validate()
            if (err != nil) != test.expectError {
                t.Errorf("Validate() = %v, want %v", err, test.expectError)
            }
            if err != nil {
                t.Errorf("Validate() error = %v, want %v", err, test.errorMsg)
            }
        })
    }
}
```

```

    FemaleEmployees: 2,
  },
  expectError: false,
},
{
  name: "Empty business name",
  request: CreateBusinessRequest{
    Name: "",
    Address: "Main Bazaar, Swat",
    District: "Swat",
    MaleEmployees: 1,
    FemaleEmployees: 0,
  },
  expectError: true,
  errorMsg: "business name is required",
},
{
  name: "Invalid district out of scope",
  request: CreateBusinessRequest{
    Name: "Khyber Tech Solutions",
    Address: "Main Bazaar, Swat",
    District: "Peshawar", // not in allowed list
    MaleEmployees: 1,
    FemaleEmployees: 0,
  },
  expectError: true,
  errorMsg: "business location district is out of allowed scope",
},
{
  name: "Negative employee counts",
  request: CreateBusinessRequest{
    Name: "Khyber Tech Solutions",
    Address: "Main Bazaar, Swat",
    District: "Swat",
    MaleEmployees: -1,
    FemaleEmployees: 0,
  },
  expectError: true,
  errorMsg: "employee counts cannot be negative",
},
}

// Iterate over the table
for _, tt := range tests {
  t.Run(tt.name, func(t *testing.T) {
    err := tt.request.Validate()

    if tt.expectError {
      if err == nil {
        t.Errorf("expected error containing %q, got nil", tt.errorMsg)
      } else if err.Error() != tt.errorMsg {
        t.Errorf("expected error message %q, got %q", tt.errorMsg, err.Error())
      }
    } else {
      if err != nil {
        t.Errorf("unexpected error: %v", err)
      }
    }
  })
}

```

## Example 2: Automatic Git Bisect Session

To find a commit that broke a test automatically:

```

# Start bisect session
git bisect start

# Mark current HEAD as bad
git bisect bad

# Mark a known good tag or commit hash (e.g., release-v1.0.0)
git bisect good release-v1.0.0

# Run git bisect automatically using a test script
git bisect run go test ./internal/business -run TestCreateBusinessRequest_Validate

```

Git will iterate through the history, run the test, and output the first bad commit.

## Mastery Check

You understand this chapter when you can:

- Write table-driven Go tests.
- Test authenticated HTTP handlers.
- Detect API contract regressions.
- Reproduce a bug with a failing test before fixing it.
- Use Git history to find when behavior changed.

# Chapter 16: Deployment, Release Git, and Mastery Roadmap

## Purpose

The final stage is operating the rewritten application: build artifacts, containers, migrations, environment configuration, release tags, and rollback strategy.

## Theoretical Background

### Multi-Stage Container Builds

Docker images are built layer-by-layer. Every instruction in a `Dockerfile` (e.g., `RUN`, `COPY`) adds a new layer to the image.

- **Cache Optimization:** By copying dependency files (like `go.mod` and `go.sum`) first and running `go mod download`, Docker caches the dependencies. Source code changes will not trigger re-downloading dependencies, speeding up builds.
- **Multi-Stage Builds:** Allow you to separate compile environments from deployment runtimes. You compile the Go app inside a heavy, tool-rich image (`golang:1.22`), then copy only the compiled binary into a minimal execution runtime image (like `alpine` or `scratch`).
- This reduces the final production image size from ~1GB down to ~30MB, eliminating build tools and reducing the security attack surface.

```
[ Stage 1: Build Env ] ---> Compiles Go code ---> (Server Binary)
                                     | (Copy Binary)
                                     v
[ Stage 2: Runtime Env ] <----- [ Alpine / Scratch Image ] (Minimal Size)
```

### Semantic Versioning (SemVer)

Releases are versioned using a standard three-part format: `MAJOR.MINOR.PATCH` (e.g., `v1.4.2`):

- **MAJOR:** Increment when you make incompatible API changes.
- **MINOR:** Increment when you add functionality in a backwards-compatible manner.
- **PATCH:** Increment when you make backwards-compatible bug fixes.

### Git Flow Branching Model

A structured branching workflow facilitates organized software development and releases:

- **main:** Stores production-ready code. Every commit here matches a tagged release.
- **develop:** Integration branch for features.
- **feature/ branches:** Short-lived branches created from `develop` to work on single features, merged back via pull requests.
- **release/ branches:** Created from `develop` when preparing for a release. Only bug fixes are allowed here before merging into `main` and `develop`.
- **hotfix/ branches:** Branches cut directly from `main` to address critical production issues, merged back to `main` and `develop`.

### External Resources

- Semantic Versioning 2.0.0 Specification
- Docker Documentation: Best Practices for Writing Dockerfiles
- Atlassian Git Tutorials: Git Flow Workflows

## Deployment Units

The system has:

- PostgreSQL 15.
- Redis 7.
- Go backend.
- Go worker or compatible worker process.
- Vue frontend served by nginx.
- Adminer or pgAdmin for database inspection.

## Release Git

Use tags:

```
git tag -a v0.1.0 -m "First Go API compatibility milestone"
git push origin v0.1.0
```

Use release branches when stabilizing:

```
git checkout -b release/v1.0.0
```

Use hotfix branches from the release tag:

```
git checkout -b hotfix/login-blocked-users v1.0.0
```

## Rollback Strategy

A rollback plan includes:

- Previous container image tag.
- Database migration rollback or forward-fix plan.
- Preserved environment variables.
- Redis cache invalidation.
- Smoke tests after rollback.

Never deploy schema changes casually. Database compatibility is harder to roll back than code.

## Practical Examples

### Example 1: Multi-Stage Production Dockerfile for Go Backend

This Dockerfile compiles a statically-linked Go binary in a build stage and runs it inside a minimal Alpine container containing no compilers or build tools:

```
# STEP 1: Build phase
FROM golang:1.22-alpine AS builder

# Install certificates and git
RUN apk update && apk add --no-cache git ca-certificates

WORKDIR /app

# Copy dependency manifests first to leverage Docker caching
COPY go.mod go.sum ./
```

```

RUN go mod download

# Copy application source files
COPY . .

# Compile binary statically, disabling CGO and targetting OS/Arch
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags="-w -s" -o server ./cmd/server

# STEP 2: Minimal Runtime phase
FROM alpine:3.19

# Import certificates to enable secure outbound HTTPS calls
COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=builder /app/server /usr/local/bin/server

# Create a non-root group and user for security isolation
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 8080

ENTRYPOINT ["/usr/local/bin/server"]

```

## Example 2: Dockerfile and Nginx setup for Vue Frontend

This multi-stage Dockerfile builds the static Vue asset bundle and serves it using Nginx, including custom routing fallbacks for single page applications:

```

# STEP 1: Node Build phase
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# STEP 2: Nginx Web Server phase
FROM nginx:1.25-alpine
COPY --from=builder /app/dist /usr/share/nginx/html

# Replace default configuration with one optimized for SPA routers
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Example supporting config file (nginx.conf) to ensure client-side routing fallback:

```

server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        # Direct all URI requests to index.html for client-side routing
        try_files $uri $uri/ /index.html;
    }

    # Proxy API calls to Go backend server
    location /api {
        proxy_pass http://sme_backend_app:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

```

## Mastery Roadmap

To master Go:

- Read standard library HTTP code.

- Practice contexts, interfaces, errors, and tests.
- Profile slow report queries.
- Use `go test -race`.
- Refactor only after tests protect behavior.

To master Vue:

- Build small components with clear props and emits.
- Keep API calls centralized.
- Manage form state explicitly.
- Test route guards and complex conditional forms.
- Treat accessibility and bilingual layout as core requirements.

To master Git:

- Commit in small units.
- Read diffs before every commit.
- Use branches intentionally.
- Practice conflict resolution.
- Tag releases.
- Learn bisect for regression hunting.

## Final Capstone

The capstone is complete when:

- All documented `/api` endpoints exist in Go.
- Existing Vue screens work against the Go backend.
- New Vue screens reproduce the applicant and admin workflows.
- PostgreSQL schema matches the original.
- Redis caching and access control work.
- HFC scoring and approval logs work.
- S3 uploads work.
- Email jobs are queued.
- Tests cover critical workflows.
- A release tag marks the milestone.

# Chapter 17: Go Beginner Primer Through Portal Features

## Purpose

This chapter is the missing beginner bridge. If you are new to Go, do not start by memorizing syntax. Start by asking: "Where will this Go concept appear in the PEACE SME portal?" Every concept below is paired with a real feature you will build.

## How Go Feels Different From Python or JavaScript

Go is compiled, statically typed, explicit, and intentionally small. That means:

- The compiler checks many mistakes before the program runs.
- You must say what shape data has.
- Errors are returned as values instead of thrown as exceptions.
- Packages are simple directories.
- Concurrency is built into the language through goroutines and channels, but web applications still need careful design.
- Formatting is standardized with `gofmt`.

In Flask, it is common to pass dictionaries around. In Go, you usually define structs. In JavaScript, a field can silently change type. In Go, `GrantRequired float64` cannot suddenly become "five hundred thousand" unless you explicitly parse it.

## Concept Map

| Go concept              | Portal task                                             | Why it matters                                 |
|-------------------------|---------------------------------------------------------|------------------------------------------------|
| Variables and constants | allowed districts, status names, cache TTLs             | Avoid magic strings scattered through handlers |
| Structs                 | User, Business, Grant, AdminUser                        | Model request, response, and database shapes   |
| Pointers                | services, repositories, large structs                   | Share dependencies without copying them        |
| Methods                 | UserService.Login, GrantRepository.FindByUserID         | Attach behavior to a type                      |
| Interfaces              | EmailQueue, Storage, TokenSigner                        | Swap real services for test fakes              |
| Slices                  | lists of documents, financed items, admins              | Represent ordered collections                  |
| Maps                    | allowed country codes, translations, filter allow-lists | Fast lookup by key                             |
| Errors                  | config parsing, SQL failures, validation                | Make failure explicit and testable             |
| Context                 | request cancellation, identity, database queries        | Carry request lifecycle through layers         |
| Goroutines              | background workers, email sending, HFC jobs             | Run work outside the request path              |
| Testing                 | auth rules, grant validation, reports                   | Protect compatibility while rewriting          |

## Variables, Constants, and Types

Theory: A variable stores a value. A constant stores a value known at compile time. Go types are strict.

Application parallel: allowed districts should be constants or a package-level list:



```
var AllowedDistricts = map[string]bool{
    "Swat":      true,
    "Shangla":   true,
    "Upper Dir": true,
    "Upper Chitral": true,
    "Lower Chitral": true,
}
```

Beginner rule: use constants for stable single values and maps/slices for groups.

```
const StatusBlocked = "blocked"
const StatusUnblocked = "unblocked"
```

Project task: create `internal/domain/constants.go` and put user statuses, grant statuses, HFC statuses, risk levels, and allowed districts there.

Git task:

```
git checkout -b chapter-17-domain-constants
git add internal/domain
git commit -m "Add domain constants for portal workflow"
```

## Structs

Theory: A struct groups named fields into one type. This is how Go represents application data.

Application parallel: the login response has a specific shape:

```
{ "token": "jwt", "user_id": 42, "language": "urdu" }
```

Model it:

```
type LoginResponse struct {
    Token    string `json:"token"`
    UserID   int64  `json:"user_id"`
    Language string `json:"language"`
}
```

The JSON tags are not optional decoration. They preserve the existing frontend contract. Without `json:"user_id"`, Go would output `UserID`, which would break code expecting `user_id`.

Project task: define request and response structs for:

- user login
- admin login
- business create/update
- grant create/update
- paginated report responses

Beginner mistake to avoid: do not use one giant struct for everything. A database row, a create request, and an API response often have different fields.

## Pointers

Theory: A pointer stores the address of a value. Go passes function arguments by value, so without pointers you copy values.

Application parallel: services hold dependencies:

```
type UserService struct {
    users *UserRepository
    tokens *TokenService
}
```

You pass pointers because:

- repositories may contain a database pool;

- services should share the same dependency instance;
- methods can avoid copying large values.

Project task: create constructors:

```
func NewUserService(users *UserRepository, tokens *TokenService) *UserService {
    return &UserService{users: users, tokens: tokens}
}
```

Beginner rule: use values for small immutable data. Use pointers for services, repositories, large structs, and anything that must be modified.

## Methods

Theory: A method is a function attached to a type.

Application parallel: instead of a free function:

```
func Login(service UserService, email string, password string) {}
```

write:

```
func (s *UserService) Login(ctx context.Context, req LoginRequest) (*LoginResponse, error) {
    // login behavior
}
```

This reads naturally: "the user service logs in a user."

Project task: implement methods for:

- UserService.Login
- BusinessService.SaveProfile
- GrantService.Apply
- AdminService.SetUserStatus
- HFCService.CalculateForUser

## Interfaces

Theory: An interface describes behavior. In Go, a type satisfies an interface automatically if it has the required methods.

Application parallel: grant approval must enqueue emails. In tests, you do not want to send real Brevo emails.

```
type EmailQueue interface {
    EnqueueGrantApproved(ctx context.Context, userID int64) error
    EnqueueApprovalNotification(ctx context.Context, userID int64) error
}
```

The real implementation writes to Redis. The test implementation records calls in memory.

Project task: define interfaces at the package that uses them. If grant .Service needs email queue behavior, define the small interface in internal/grant, not a giant global interface package.

Beginner rule: accept interfaces, return concrete types.

## Slices

Theory: A slice is a dynamic list.

Application parallel: a grant has financed items:

```
type FinancedItem struct {
    Item          string `json:"item"`
    Quantity      int    `json:"quantity"`
    EstimatedCost float64 `json:"estimated_cost"`
}
```

```

}

type GrantRequest struct {
    FinancedItems []FinancedItem `json:"financed_items"`
}

```

Project task: validate that each financed item has a name, positive quantity, and positive cost.

## Maps

Theory: A map is a key-value lookup table.

Application parallel: report sorting must be safe. Never put raw `sort_by` directly into SQL.

```

var applicantSortColumns = map[string]string{
    "created_at": "u.created_at",
    "district":   "b.business_location_district",
    "status":     "u.status",
}

```

Project task: use map allow-lists for:

- report sort columns
- allowed countries
- allowed districts
- admin roles
- HFC risk bands if helpful

## Errors

Theory: Go functions return errors. You must check them.

Application parallel: login can fail because the request body is invalid, the user is missing, the password is wrong, the user is blocked, or token signing failed.

Do not collapse all errors too early. Inside the service, keep meaningful errors:

```

var ErrInvalidCredentials = errors.New("invalid credentials")
var ErrUserBlocked = errors.New("user is blocked")

```

The handler converts them into HTTP responses:

```

switch {
case errors.Is(err, ErrInvalidCredentials):
    httpx.WriteError(w, http.StatusUnauthorized, "invalid_credentials", "Invalid email or password")
case errors.Is(err, ErrUserBlocked):
    httpx.WriteError(w, http.StatusForbidden, "user_blocked", "Your account is blocked")
default:
    httpx.WriteError(w, http.StatusInternalServerError, "server_error", "Unexpected error")
}

```

Project task: define service-level errors for auth, validation, not found, forbidden, and conflict cases.

## Context

Theory: `context.Context` carries cancellation, timeouts, and request-scoped values.

Application parallel: when a browser cancels a report request, the Go handler's context is canceled. Database queries should receive that context so PostgreSQL work can stop too.

```

func (r *ReportRepository) Applicants(ctx context.Context, filter ApplicantFilter) ([]ApplicantRow, error) {
    rows, err := r.db.Query(ctx, query, args...)
}

```

Project task: every handler should call service methods with `r.Context()`. Every repository method should accept `ctx context.Context`.

## Goroutines and Background Work

Theory: a goroutine is a lightweight concurrent function started with `go`.

Application parallel: HFC scoring and email sending should not block the applicant's grant submission response.

Beginner warning: do not casually start goroutines inside handlers for production-critical work. If the process crashes, the goroutine disappears. Use Redis-backed jobs or an outbox table for reliable work.

Good learning progression:

- 1 Start with synchronous fake implementations.
- 2 Add interfaces.
- 3 Add Redis queue implementation.
- 4 Add worker process.
- 5 Add retry and logging.

## Testing

Theory: Go tests live in files ending with `_test.go`.

Application parallel: write table-driven tests for grant validation:

```
func TestValidateGrantRequest(t *testing.T) {
    tests := []struct {
        name    string
        req     GrantRequest
        wantErr bool
    }{
        {name: "missing amount", req: GrantRequest{}, wantErr: true},
        {name: "valid financed item", req: validGrant(), wantErr: false},
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            err := ValidateGrantRequest(tt.req)
            if (err != nil) != tt.wantErr {
                t.Fatalf("expected error=%v, got %v", tt.wantErr, err)
            }
        })
    }
}
```

Project task: create tests for:

- configuration parsing
- JWT signing and verification
- blocked user login
- grant whitelist gate
- HFC risk scoring
- report sort allow-list

## Capstone Exercise

Build a tiny vertical slice:

- 1 Define `LoginRequest` and `LoginResponse`.

- 2     Implement `UserRepository.FindByEmail`.
- 3     Implement `UserService.Login`.
- 4     Implement `POST /api/login`.
- 5     Add a Vue login form that stores `userToken`.
- 6     Commit each layer separately.

When this works, you have practiced structs, methods, pointers, interfaces, errors, context, JSON, SQL, Vue state, and Git.

# Chapter 18: Project Parallel Workbook

## Purpose

This chapter turns the book into a build plan. Each row pairs a concept with a concrete PEACE SME implementation task. Use it when you ask, "What should I build to understand this topic?"

## Parallel Task Table

| Learning topic | Theory to study                      | Backend task                            | Vue task                        | Git task                            |
|----------------|--------------------------------------|-----------------------------------------|---------------------------------|-------------------------------------|
| HTTP routing   | request methods, paths, status codes | implement /health and /api/updates      | show updates on landing page    | commit route and handler separately |
| JSON encoding  | struct tags, request decoding        | implement ReadJSON and WriteJSON        | consume JSON with Axios         | inspect diff for response names     |
| Config         | env vars, defaults, validation       | parse GRANT_APPLICATION_OPEN            | show registration closed page   | add config tests before commit      |
| Auth           | JWT, bcrypt, bearer headers          | implement /api/login                    | store userToken                 | branch feature/user-login           |
| Admin auth     | roles and permissions                | implement /api/admin/login              | store adminToken                | commit admin claims tests           |
| Middleware     | chain of handlers                    | auth, geo-block, access slot middleware | redirect unauthenticated routes | diff middleware order               |
| Database       | schema, indexes, joins               | migrate users, businesses               | dashboard fetches profile       | tag first DB milestone              |
| Transactions   | atomic multi-step writes             | approve grant and log action            | approval button                 | commit failing test then fix        |
| JSONB          | typed nested data                    | save financed_items                     | dynamic item rows               | review JSON names                   |
| Validation     | business rule enforcement            | allowed districts, contribution rules   | inline form errors              | commit validation table tests       |
| Uploads        | presigned URLs, object keys          | generate upload URL                     | document upload component       | branch feature/uploads              |
| Redis          | TTL, cache, debounce                 | cache FAQs and updates                  | FAQ drawer consumes API         | commit cache invalidation           |
| Reports        | pagination, filters, sorting         | applicant report endpoint               | admin table filters             | compare query params diff           |
| CSV            | streaming, query tokens              | full applicant export                   | export button                   | add token expiry test               |
| HFC            | deterministic scoring                | calculate score and risk                | HFC queue display               | commit rule tests one by one        |
| Workers        | durable background jobs              | enqueue HFC and email jobs              | show pending state              | tag async milestone                 |
| Bilingual UI   | translation dictionaries, RTL        | return language from login              | language switcher               | commit English/Urdu together        |
| Deployment     | containers, migrations               | Dockerfile and migration runner         | static frontend build           | tag release candidate               |

## Milestone 1: Skeleton That Teaches Go Basics

Goal: make the Go program feel understandable.

Build:

- cmd/server/main.go
- internal/config
- internal/httpx
- internal/health

- internal/app

Theory:

- packages
- exported vs unexported names
- functions
- structs
- pointers
- errors

Portal feature:

- /health proves the server runs.
- /api/updates proves a public JSON route works.

Vue parallel:

- create a landing page section that calls /api/updates.

Git:

```
git checkout -b milestone-01-server-skeleton
git commit -m "Add Go server skeleton"
git commit -m "Add public updates endpoint"
```

Done when:

- go test ./... passes.
- /health returns JSON.
- /api/updates returns an empty list or seeded rows.

## Milestone 2: Login Vertical Slice

Goal: understand web auth end to end.

Build:

- users table migration.
- UserRepository.FindByEmail.
- bcrypt password verification.
- JWT token generation.
- POST /api/login.
- Vue login page.
- route guard for /dashboard.

Theory:

- SQL query result scanning.
- nullable values.
- service errors.
- JWT claims.
- localStorage.

- Axios headers.

Portal feature:

- applicant logs in with email and password.
- blocked user receives a specific failure.
- successful login returns {token, user\_id, language}.

Git:

```
git checkout -b feature/user-login
git commit -m "Add user login repository"
git commit -m "Add JWT user token service"
git commit -m "Implement user login endpoint"
git commit -m "Add Vue user login flow"
```

Done when:

- invalid password fails.
- blocked user fails.
- valid login stores userToken.
- authenticated route sends Authorization: Bearer.

## Milestone 3: Business Profile

Goal: learn create/update workflows and validation.

Build:

- businesses migration.
- BusinessRequest struct.
- allowed district validation.
- GET /api/business.
- POST /api/business.
- PUT /api/business.
- Vue business profile form.

Theory:

- one-to-one table relation.
- unique constraints.
- validation before database writes.
- form state with reactive.

Portal feature:

- each applicant has one business profile.
- district must be in scope.
- profile can be edited.

Git:

```
git checkout -b feature/business-profile
git commit -m "Add business profile schema"
git commit -m "Implement business profile service"
git commit -m "Add Vue business profile form"
```



Done when:

- creating twice does not create duplicate rows.
- invalid district is rejected.
- form loads existing profile.

## Milestone 4: Grant Form and Whitelist Gate

Goal: master larger payloads and workflow rules.

Build:

- grants migration.
- grant\_access\_whitelist migration.
- grant request validation.
- GET /api/grant.
- POST /api/grant.
- PUT /api/grant.
- GET /api/grant-status.
- Vue multi-section grant form.

Theory:

- JSONB fields.
- slices of structs.
- conditional validation.
- workflow gates.
- HTTP 403 vs 409 vs 422.

Portal feature:

- user can apply only when whitelisted if GRANT\_REQUIRE\_SELECTION=1.
- one user has one grant.
- financed items repeat dynamically.
- SRSP relatives are conditional rows.

Git:

```
git checkout -b feature/grant-application
git commit -m "Add grant schema and models"
git commit -m "Implement grant access whitelist"
git commit -m "Implement grant application endpoint"
git commit -m "Add Vue grant application form"
```

Done when:

- non-whitelisted user cannot submit.
- whitelisted user can submit.
- JSONB fields round-trip.
- Vue review page displays the same data.

## Milestone 5: Admin Reports

Goal: learn real data querying.

Build:

- admin token verification.
- applicant report endpoint.
- filters, sort allow-list, pagination.
- admin report table.
- CSV export query token.

Theory:

- joins.
- count queries.
- pagination.
- SQL injection prevention.
- streaming responses.

Portal feature:

- admins can filter applicants by district, sector, status, language, gender, and search.

Git:

```
git checkout -b feature/admin-reports
git commit -m "Add applicant report query"
git commit -m "Add report filters and pagination"
git commit -m "Add Vue admin applicant report"
```

Done when:

- unauthorized users cannot access reports.
- invalid sort field is rejected or replaced by a safe default.
- pagination response shape is exactly {data, total, page, per\_page}.

## Milestone 6: HFC and Approval

Goal: learn deterministic business logic, auditability, and side effects.

Build:

- HFC scoring rules.
- hfc\_evaluations writes.
- admin HFC queue.
- approval check endpoint.
- approval transaction.
- email enqueue interface.

Theory:

- pure functions.
- risk bands.
- audit logs.
- transactions.

- interfaces for side effects.

Portal feature:

- grant submission triggers HFC scoring.
- approver can approve grants.
- shadow mode means HFC does not block approval.

Git:

```
git checkout -b feature/hfc-approval
git commit -m "Add deterministic HFC scoring"
git commit -m "Store HFC evaluations"
git commit -m "Implement grant approval transaction"
```

Done when:

- scoring tests prove each rule.
- approval writes both grant and log.
- non-approver cannot approve.
- shadow mode behavior is tested.

## Milestone 7: Production Readiness

Goal: make it runnable and maintainable.

Build:

- Dockerfile.
- migration runner.
- Redis cache.
- worker process.
- S3 upload integration.
- structured logs.
- smoke tests.

Theory:

- deployment units.
- process lifecycle.
- graceful shutdown.
- observability.
- rollback.

Portal feature:

- full applicant workflow and admin workflow run on the Go + Vue stack.

Git:

```
git checkout -b release/v0.1.0
git tag -a v0.1.0 -m "First Go and Vue portal milestone"
```

Done when:

- backend starts from clean environment.

- migrations apply once.
- Vue build succeeds.
- critical tests pass.
- release tag exists.

# Chapter 19: Vue and Git Practice Lab

## Purpose

This lab makes Vue and Git practical. You will build frontend behavior that matches the Go API while using Git to protect every step.

## Vue Beginner Concepts by Portal Feature

| Vue concept        | Portal feature                      | What you learn                   |
|--------------------|-------------------------------------|----------------------------------|
| ref                | login email/password fields         | single primitive reactive values |
| reactive           | business profile form               | object-shaped form state         |
| computed           | Urdu/English label selection        | derived state                    |
| watch              | filters triggering report reloads   | reacting to changes              |
| onMounted          | fetch dashboard data                | lifecycle fetching               |
| props              | reusable form input                 | parent-to-child data             |
| emits              | modal close, upload complete        | child-to-parent events           |
| router guards      | protected dashboard/admin pages     | client-side auth flow            |
| composables        | useAuth, useLanguage, usePagination | reusable stateful logic          |
| Axios interceptors | bearer auth headers                 | API integration                  |

## Lab 1: Login Page

Theory:

- ref is best for simple values.
- v-model binds input value to state.
- submit handlers call API functions.

Portal task:

- build UserLogin.vue.
- call POST /api/login.
- store userToken.
- redirect to /dashboard.

Example:

```
<script setup>
import { ref } from 'vue'
import { useRouter } from 'vue-router'
import api from '../api/client'

const router = useRouter()
const email = ref('')
const password = ref('')
const error = ref('')
const loading = ref(false)

async function login() {
  error.value = ''
  loading.value = true
  try {
    const { data } = await api.post('/login', {
      email_address: email.value,
```

```

        password: password.value,
      })
      localStorage.setItem('userToken', data.token)
      router.push('/dashboard')
    } catch (err) {
      error.value = 'Invalid login or blocked account'
    } finally {
      loading.value = false
    }
  }
}
</script>

```

Git practice:

```

git checkout -b vue-user-login
git add frontend/src/views/UserLogin.vue
git commit -m "Add Vue user login page"

```

## Lab 2: Business Profile Form

Theory:

- reactive is useful for form objects.
- select inputs should store backend-compatible values.
- frontend validation improves UX, but backend validation remains authoritative.

Portal task:

- build `SmeBusinessProfile.vue`.
- fetch existing data with GET `/api/business`.
- create with POST `/api/business`.
- update with PUT `/api/business`.

State shape:

```

const form = reactive({
  name_of_business: '',
  business_registration_number: '',
  business_location_district: '',
  business_sector: '',
  male_employees: 0,
  female_employees: 0,
})

```

Parallel Go task:

- `BusinessRequest` struct has matching JSON tags.
- Go validates allowed districts.
- repository uses INSERT or UPDATE.

Git practice:

```

git checkout -b vue-business-profile
git commit -m "Add business profile form state"
git commit -m "Connect business profile form to API"

```

## Lab 3: Grant Application Dynamic Sections

Theory:

- arrays in Vue are reactive when stored inside reactive.
- conditional rendering uses `v-if`.
- repeated rows use `v-for`.

Portal task:

- add financed item rows.
- show cash fields only when contribution includes cash.
- show in-kind fields only when contribution includes in-kind.
- show relatives table only when `has_srsp_relative` is true.

Example:

```
function addFinancedItem() {
  form.financed_items.push({
    item: '',
    quantity: 1,
    estimated_cost: null,
  })
}
```

Parallel Go task:

- decode `financed_items` into `[]FinancedItem`.
- validate each row.
- store as JSONB.

Git practice:

```
git checkout -b vue-grant-form
git commit -m "Add grant financed items UI"
git commit -m "Add conditional grant contribution fields"
git commit -m "Connect grant form to API"
```

## Lab 4: Admin Report Table

Theory:

- tables need query state: page, per page, filters, sort.
- watch can reload data when filters change.
- query params should mirror backend filter names.

Portal task:

- build applicant report table.
- support page, per\_page, district, sector, status, search, sort\_by, sort\_dir.

Parallel Go task:

- parse query params into `ApplicantReportFilter`.
- use SQL allow-list for sort fields.
- return `{data, total, page, per_page}`.

Git practice:

```
git checkout -b vue-admin-report
git commit -m "Add admin applicant report table"
git commit -m "Connect admin report filters"
```

## Git Lab: Learn by Reviewing Your Own Work

Before every commit:

```
git status
git diff
git add <files>
```

```
git diff --staged
git commit -m "Specific behavior"
```

Review questions:

- Did I include unrelated files?
- Did I accidentally commit generated output?
- Does the commit message describe behavior?
- Could this commit be reverted safely?
- Are tests or screenshots needed?

## Git Lab: Feature Branch to Merge

```
git checkout main
git checkout -b feature/grant-form
# build the feature
git add frontend/src/views/SmeGrantApplication.vue
git commit -m "Add grant application form"
git checkout main
git merge feature/grant-form
```

Conflict practice:

- 1 Create two branches that edit the same route list.
- 2 Merge one branch.
- 3 Merge the second branch.
- 4 Resolve the route conflict manually.
- 5 Run tests.
- 6 Commit the merge.

## Capstone Lab

Build one complete user story:

- 1 User logs in.
- 2 User opens dashboard.
- 3 User creates business profile.
- 4 Admin marks applicant eligible.
- 5 Admin whitelists user for grants.
- 6 User submits grant.
- 7 HFC score appears for admin.
- 8 Approver approves grant.

For every step, write down:

- Vue route.
- API endpoint.
- Go handler.
- Go service method.
- Repository query.
- Database table.



● Git commit.

This is the real learning loop. The concepts become durable when you can trace a button click all the way to SQL and back.